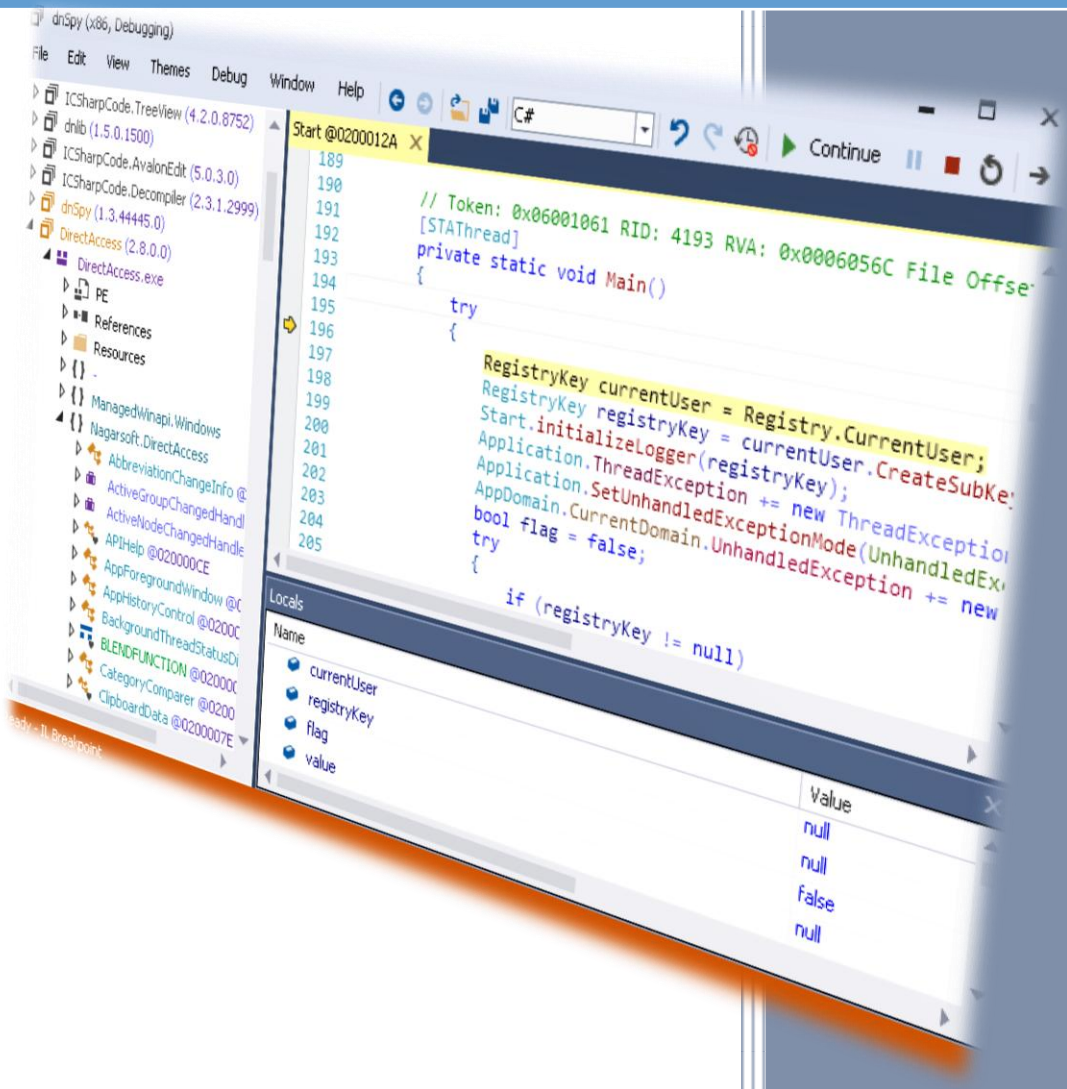
2015

Jugando con un .NET en dnSpy



Snat
[CrackSLatinoS]

Jugando con un .NET en dnSpy

CrackSLatinoS

Noviembre 2015

1. Contenido

1. Contenido 1
2. Introducción 2
3. Material necesario 3
4. Comienza el estudio 4
5. Código IL ? 27
6. Todos los caminos llevan a Roma 41
7. En busca de los Metadatos perdidos 46
8. Notas Finales 51

2. Introducción

En este texto vamos a solucionar de diferente modo el tema del dumpeado de DirectAccess 2.8.0.0 con el que trabajamos en el tutorial Desempacando Themida en un .NET (Teoría número 1571 de nuestro querido grupo CrackSLatinoS). Todo esto inicia con el envío de una herramienta por parte de sequeyo a la lista, hilo:

- `{CrackSLatinoS}: Nueva herramienta dnSpy pana los .NET`

En estos días habíamos estado trabajando con los .NET y sequeyo se tomó la molestia de enviar esta herramienta. Él la utiliza bastante y quería compartirla con el resto del grupo: bonito detalle que por su puesto le agradecemos, su nombre: dnSpy 1.3.44445.0

Entonces la probé y la verdad me encantó desde el primer momento, lo primero que pensé fue lo sencillo que se hacía trabajar con los .NET, hasta ahora herméticos y casi desconocidos.

Si abrirmos el programa y miramos en la ventanita About del programa vemos esta información:

- `dnSpy version 1.3.44445.0`
- `ILSpy Decompiler version 2.3.1`

En este texto nos centraremos principalmente en el uso de esta herramienta. Por mi parte he de decir que es la primera vez que voy a utilizarla, hasta el día de ayer no la había ni visto. Aun así, como se podrá comprobar, su uso es muy sencillo e intuitivo.

El objetivo del texto es dar nuestros primeros pasos con dnSpy, y encontrar al menos un camino de lograr que nuestro dumpeado corra, sin utilizar código externo al propio binario (Loaders y demás), SÓLO con dnSpy ese es el desafío.

Durante el desarrollo del texto es posible que hagamos un pequeño repaso a estructuras del .NET con el objetivo de obtener una mejor visión del mismo.

Sin más dilación vamos a ello..

3. Material necesario

El software con el que trabajaremos será Direct Access versión 2.8.0.0

www.nagarsoft.com/directaccess/

El dumpeado lo obtuvimos en el tutorial anterior (Teoría número 1571). Aunque también se puede obtener un dumpeado con MegaDumper 1.0 y Universal Fixer 1.0, ambas de SnD.

También utilizaremos estas siguientes herramientas:

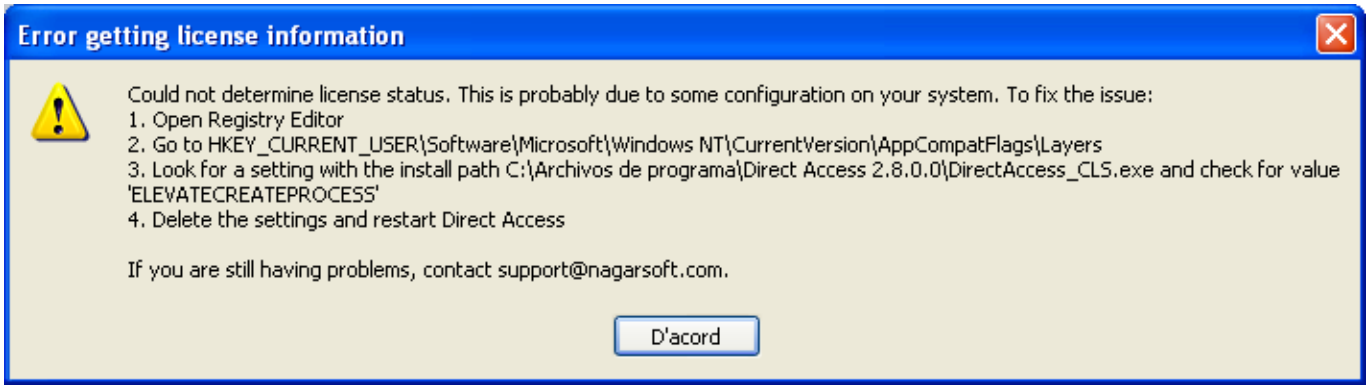
- ✓ **dnSpy** 1.3.44445.0
- ✓ OllyDbg 1.10 de Oleg Yuschuk (opcional)
- ✓ LordPE Deluxe de yoda (opcional)

Estas herramientas las podemos encontrar en el web de Ricardo Narvaja:

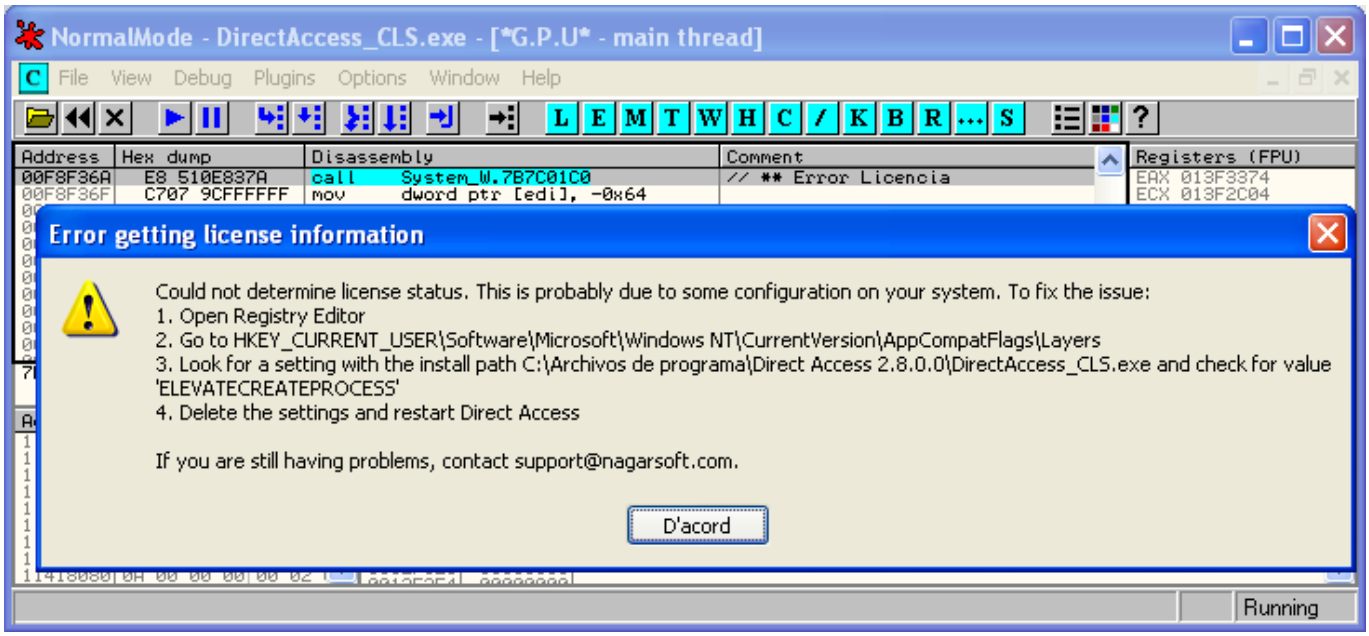
http://www.ricardonarvaja.info/WEB/

4. Comienza el estudio

Comencemos. Tenemos el dumpeado que si ejecutamos muestra este cartel:



Sabemos (del texto de referencia anterior) que este cartel se muestra en esta dirección:

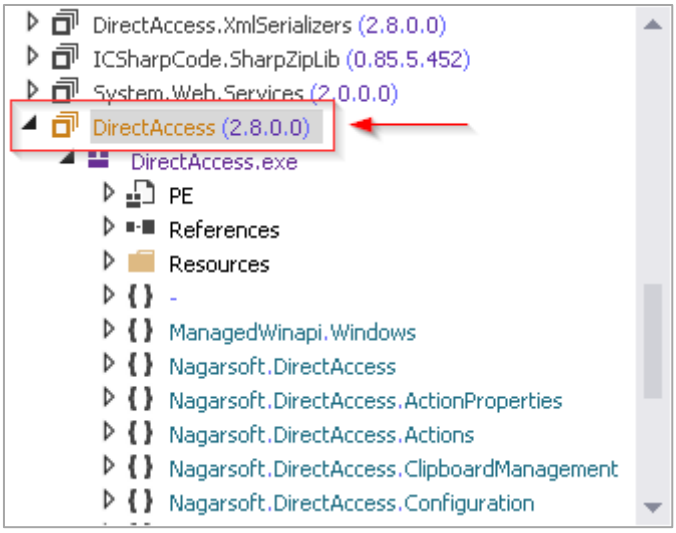


Esa call de 0xF8F36Ah es la responsable del cartel, la manera de llegar hasta aquí la vimos en el texto citado, y que fue colocando un BP con <F2> en la api user32.DestroyWindow y rastreando los *call padre* hasta localizar la dirección de la imagen anterior.

Este será nuestro punto de partida. He de decir que estoy escribiendo el texto a la par que elijo la estrategia a seguir y los contenidos que va a tener éste, es probable que terminemos en un entorno de desarrollo escribiendo código, nunca se sabe, la idea es sacar el máximo partido el tutorial.

Tenemos fijado el punto de partida y el punto de llegada (el objetivo). Lo que es variable es el camino que vamos a seguir para llegar a la META.

Abrimos dnSpy y cargamos nuestro dumpeado que en mi caso quedó con el nombre DirectAccess_CLS.exe, y deberemos ver algo así:



Esta es nuestra primera toma de contacto con dnSpy, en el panel izquierdo se ven otros Módulos cargados, además del nuestro y que estamos estudiando: DirectAccess (2.8.0.0).

En el panel derecho se muestra información del mismo:

```
DirectAccess (2.8.0.0)   DateTime @02000032
1  // C:\Archivos de programa\Direct Access 2.8.0.0\DirectAccess_CLS.exe
2  // DirectAccess, Version=2.8.0.0, Culture=neutral, PublicKeyToken=null
3
4  // Entry point: Nagarsoft.DirectAccess.Start.Main   1
5
6  using System;
7  using System.Reflection;
8  using System.Runtime.CompilerServices;
9  using System.Runtime.InteropServices;
10                                                      3
11 [assembly: AssemblyVersion("2.8.0.0")]
12 [assembly: AssemblyCompany("Nagarsoft")]
13 [assembly: AssemblyConfiguration("")]
14 [assembly: AssemblyCopyright("Copyright © 2005-2010. Nagarsoft. All rights reserved.")]
15 [assembly: AssemblyDescription("Direct Access")]
16 [assembly: AssemblyProduct("Direct Access")]
17 [assembly: AssemblyTitle("Nagarsoft Direct Access")]
18 [assembly: AssemblyTrademark("")]
19 [assembly: CompilationRelaxations(8)]
20 [assembly: RuntimeCompatibility(WrapNonExceptionThrows = true)]
21 [assembly: ComVisible(false)]
22
```

Módulos Importados 2

Se aprecian tres bloques por así decirlo:

1. Aunque esté comentado (no afecta) y es como si no estuviera, hace referencia al punto de entrada del Assembly: Nagarsoft.DirectAccess.Start.Main
2. Hace referencia a las clases (externas) que se van a utilizar con su espacio de nombres (en este caso System). Cualquier clase que vaya a ser utilizada debe ser listada aquí. Viene a ser algo similar a la directiva *#include* de C/C++ o *import* de java.
3. El el archivo MANIFEST del Assembly.

Estamos viendo la información tal como nos la mostraría .NET Reflector.

Pero hay que tener claro que *dnSpy* no se limita a mostrar el código estático y nos provee además la opción de depuración. Por ahora estamos haciendo un repaso superficial. Si vamos a la parte del árbol Nagarsoft.DirectAccess encontramos WinLicenseSDK:

En la parte derecha encontramos más información de la misma:

WinLicenseSDK @0200014D | DateTime @02000032

```
1  using System;
2  using System.Globalization;
3  using System.Runtime.InteropServices;
4  using System.Text;
5
6  namespace Nagarsoft.DirectAccess   1
7  {
8      // Token: 0x0200014D RID: 333
9      internal class WinLicenseSDK   2
10     {
11         // Token: 0x06001132 RID: 4402 RVA: 0x00062454 File Offset: 0x00062454
12         public static int DisableCurrentLicense()   3
13         {
14             StringBuilder stringBuilder = new StringBuilder(100);
15             WinLicenseSDK.GetEnvironmentVariable("WLRegDisableKeyCurrentInstance", stringBuilder, 100);
16             int num = Convert.ToInt32(stringBuilder.ToString());
17             Logger.Main.LogMessage("WLRegDisableKeyCurrentInstance: " + stringBuilder);
18             if (num == 1)
19             {
20                 Logger.Main.LogMessage("Current Key Disabled for the instance");
21             }
22             return num;
23         }
24
25         // Token: 0x06001136 RID: 4406 RVA: 0x000625D0 File Offset: 0x000625D0
26         public static int EvaluationDaysLeft()   3
27         {
28             StringBuilder stringBuilder = new StringBuilder(100);
29             WinLicenseSDK.GetEnvironmentVariable("WLTrialDaysLeft", stringBuilder, 100);
30             int num;
31             int.TryParse(stringBuilder.ToString(), out num);
32             Logger.Main.LogInt("days left from WLTrialDaysLeft", num);
33             if (num == -1)
34             {
35                 WinLicenseSDK.GetEnvironmentVariable("WLTrialDateDaysLeft", stringBuilder, 100);
36                 int.TryParse(stringBuilder.ToString(), out num);
37                 Logger.Main.LogInt("days left from WLTrialDateDaysLeft", num);
38             }
39             return num;
40         }
41
42         // Token: 0x0600112B RID: 4395                                                    4
43         [DllImport("Kernel32.dll", CallingConvention = CallingConvention.StdCall, SetLastError = true)]
44         public static extern int GetEnvironmentVariable(string lpName, StringBuilder lpBuffer, int nSize);
45
```

Veamos qué podemos extraer de todo ello:

1. Espacio de nombres: Nagarsoft.DirectAccess
2. Nombre de la clase: WinLicenseSDK. Hay que decir que las clases también pueden ser de diferentes tipos: abstracta, estática, etc., y también se pueden clasificar por ubicación: Interna (inner class) o externa (outter class). En este caso concreto se trata de una clase interna (internal).
3. Éstos son métodos, en el primer caso se define el método con con nombre *DisableCurrentLicense* con modificadores public que determina la accesibilidad que tendrá el método y static que indica que es un método de clase. Vamos a tratar de explicar qué significado tiene y qué implicaciones en palabras llanas:

   ✓ En programación orientada a objetos (el caso actual), para acceder a los métodos definidos en una clase se utiliza un objeto, esto es, por ejemplo:

```
1  String s = new String("punto NET");
2  s = s.concat(" para CLS!");
```

   Se define un Objeto tipo String con la cadena "punto NET" y después se modifica agregándole la cadena " para CLS". Lo importante es darse cuenta del modo en que está utilizando el método concat(String), hace lo siguiente:

```
objeto.método( parámetros );
```

   Se utiliza la referencia al objeto tipo String (con nombre *s* en el ejemplo), seguido de un punto y el nombre del método. A esta nomenclatura se le denomina notación de punto y se utiliza para acceder a los métodos de la Clase necesaria a través de la referencia de un objeto de esta clase. Recordemos que un objeto se crea a partir de una clase (operador new, newobj, etc.,).

   Lo que hace el modificador *static* es cambiar las reglas de juego: Ahora NO sería necesaria la intervención del objeto para utilizar el método, basta con utilizar el nombre de Clase en su lugar. Podríamos por ejemplo, hacer lo siguiente:

```
1  String s = new String("punto NET");
2  s = String.concat(" para CLS!");
```

   SIEMPRE y cuando el método concat esté definido como STATIC dentro de la clase String, este NO es el caso tal y como se puede comprobar:

```
 */
public String concat(String str) {
    int otherLen = str.length();
    if (otherLen == 0) {
        return this;
    }
    int len = value.length;
    char buf[] = Arrays.copyOf(value, len + otherLen);
    str.getChars(buf, len);
    return new String(buf, true);
}
```

Se trata del código fuente de la clase *java.lang.String* de java. El método NO es static por lo que es obligatorio y necesario acceder a este método a través de un objeto.

Además, en el método *DisableCurrentLicense* se puede apreciar el término *int* entre el static y el nombre del mismo en su definición, esto indica que se retorna un valor de tipo entero, es decir, determina el tipo de retorno que tiene el método en caso de tenerlo, de lo contrario estaríamos obligados a indicarlo mediante la palabra reservada *void*.

```
[public | private | …] void nombre_Metodo( {Parámetro,[Parámetro]} )
```

Se aplican los mismos criterios para el segundo método de la imagen remarcada con el valor 3: EvaluationDaysLeft().

4. Atributo DllImport: Este es utilizado para invocar una función nativa, de esta manera de cara al compilador declaramos una función ubicada en una librería externa (que bien puede ser user32, kernel32, etc.), a estas funciones se les denomina *unmanaged functions* debido a que no son manejadas en su ejecución por parte del CLR. En el lado opuesto se encuentra el *managed code*, que es generado cuando el compilador traduce el código escrito (C#, VB, F# o similar) a código CIL. Asimismo es posible escribir código IL directamente y ensamblarlo con la utilidad ILASM.EXE, justo la contraria a ILDASM.EXE. Esto nos lo apuntamos :)

   Después define la convención utilizada para el paso de parámetros, esto afecta al reajuste de pila al salir de la función, si elegimos el modo `__cdecl` habrá que reajustarla manualmente:

```
//  caso __cdecl
push param1
push param2
push param3
call funcion
add  esp, 0x0Ch    // reajuste de pila
```

   En caso de establecer el modo `__stdcall` no será necesario el reajuste de ESP:

```
//  caso __stdcall
push param1
push param2
push param3
call funcion
// NO es necesario el reajuste de pila
```

   Lo tendremos en cuenta también. Asimismo hay una tercera opción `__fastcall` que por lo visto no se utiliza demasiado.

   Por último hace alusión a GetLastError con asignación a valor true (activo), y que básicamente afecta al uso de la llamada a la función GetLastError al retornar de la llamada a la función declarada, GetEnvironmentVariable en este caso, según he ido viendo esto lo hace para preservar este valor en las sucesivas llamadas a otras funciones, invocando a la función tras ejecutar la api nativa en cuestión, y cuyo valor cotejará más adelante mediante otra llamada a Marshall.GetLastWin32Error para obtener el valor previamente salvado por el CLR.

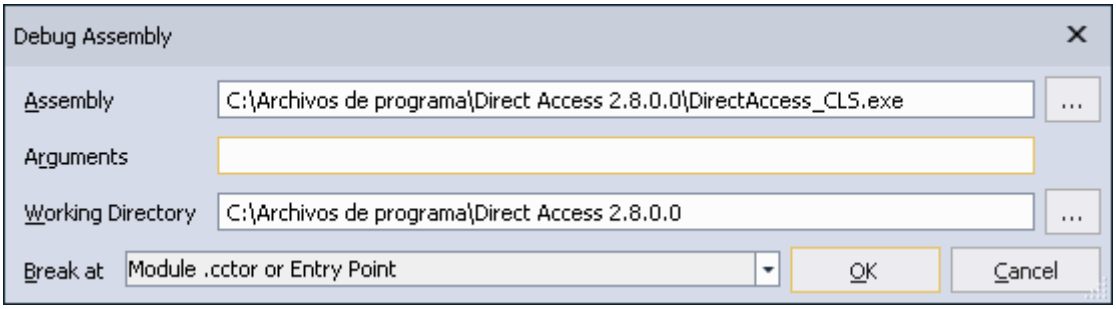
Más o menos es lo que se desprende del código anterior.

Al final de la clase se encuentran las definiciones de las variables y constantes, así como una enumeración, tal como se ve en la siguiente imagen:

```
WinLicenseSDK @0200014D
464         // Token: 0x04000CBF RID: 3263
465         public const int wlTrialRuntimeExpired = 4;
466
467         // Token: 0x04000CD3 RID: 3283
468         public const int wlWrongLicenseExp = 8;
469
470         // Token: 0x04000CD4 RID: 3284
471         public const int wlWrongLicenseHardware = 9;
472
473         // Token: 0x04000CDA RID: 3290
474         private StringBuilder _bufferVarValue = new StringBuilder(100);
475
476         // Token: 0x0200014E RID: 334
477         public enum enumGetStatus
478         {
479             // Token: 0x04000CF6 RID: 3318
480             trialStatus,
481             // Token: 0x04000CF7 RID: 3319
482             trialExtStatus,
483             // Token: 0x04000CF8 RID: 3320
484             regStatus
485         }
486     }
487 }
488
```

Esto en cuanto a la definición de la clase.

En el texto anterior le atacamos por las variables de entorno WLRegGetStatus y WLRegGetLicenseInfo, las creábamos desde el Loader o Cargador para que el programa las encontrara y se ejecutara bien. Pero lo que más nos interesa es introducirnos en las bondades de dnSpy, vamos a depurar, pulsamos <F5> o desde el menú *Debug -> Debug an Assembly* y se abre esta ventanita:



Pulsamos <OK> y se detiene en la entrada de la función Main():

```
Start @0200012A
191 [STAThread]
192 private static void Main()
193 {
194     try
195     {
196         RegistryKey currentUser = Registry.CurrentUser;
197         RegistryKey registryKey = currentUser.CreateSubKey("SOFTWARE\\Nagarsoft\\Direct Access");
198         Start.initializeLogger(registryKey);
199         Application.ThreadException += new ThreadExceptionEventHandler(Start.Application_ThreadEx
200         Application.SetUnhandledExceptionMode(UnhandledExceptionMode.CatchException);
201         AppDomain.CurrentDomain.UnhandledException += new UnhandledExceptionEventHandler(Start.Cu
```

Esa flechita amarilla marca la instrucción actual a ejecutar (sería como el EIP), si miramos la ventanita auxiliar inferior vemos allí las variables:

Locals

Name	Value	Type
currentUser	null	Microsoft.Win32.RegistryKey/*0x010000F7*/ {Microsoft.Win32.RegistryKey/*0x0200045B*/}
registryKey	null	Microsoft.Win32.RegistryKey/*0x010000F7*/ {Microsoft.Win32.RegistryKey/*0x0200045B*/}
flag	false	bool
value	null	string
e	null	System.InvalidCastException/*0x010002BE*/ {System.InvalidCastException/*0x020000C7*/}
flag2	false	bool

Si no está visible esta ventana la podemos mostrar/ocultar desde *Debug -> Locals* o <Alt+4>. Es importante porque desde aquí se pueden editar los valores de las mismas, si pulsamos <F2> sobre cualquiera de ellas podemos modificar directamente su valor.

Coloquemos un BP al inicio del método WinLicenseSDK::GetRegistrationStatus:

GetRegistrationStatus() : int @06001131

```
1  // Nagarsoft.DirectAccess.WinLicenseSDK
2  // Token: 0x06001131 RID: 4401 RVA: 0x000623B4 File Offset: 0x000623B4
3  public static int GetRegistrationStatus()
4  {
5      StringBuilder stringBuilder = new StringBuilder(100);
6      string environmentVariable = Environment.GetEnvironmentVariable("WLRegGetStatus");
7      WinLicenseSDK.GetEnvironmentVariable("WLRegGetStatus", stringBuilder, 100);
8      Logger.Main.LogString("WLRegGetStatus", stringBuilder.ToString());
9      Logger.Main.LogString("WLRegGetStatus Managed", environmentVariable);
10     int result;
11     if (string.IsNullOrEmpty(environmentVariable))
12     {
13         result = -100;
14         Logger.Main.LogError("ERROR retrieving WLRegGetStatus");
15     }
16     else
17     {
```

Damos RUN con <F5> y paramos en el IL BreakPoint:

Start @0200012A

```
191  [STAThread]
192  private static void Main()
193  {
194      try
195      {
196          RegistryKey currentUser = Registry.CurrentUser;
197          RegistryKey registryKey = currentUser.CreateSubKey("SOFTWARE\\Nagarsoft\\Direct Access");
198          Start.initializeLogger(registryKey);
199          Application.ThreadException += new ThreadExceptionEventHandler(Start.Application_ThreadExc
200          Application.SetUnhandledExceptionMode(UnhandledExceptionMode.CatchException);
201          AppDomain.CurrentDomain.UnhandledException += new UnhandledExceptionEventHandler(Start.Cur
202          bool flag = false;
203          try
204          {
```

Ready - IL Breakpoint

Este vendría a ser el EntryPoint del Assembly.

Pulsamos nuevamente <F5> y paramos en nuestro BP:

```
WinLicenseSDK @0200014D
148         // Token: 0x06001131 RID: 4401 RVA: 0x000623B4 File Offset: 0x000623B4
149         public static int GetRegistrationStatus()
150         {
151             StringBuilder stringBuilder = new StringBuilder(100);
152             string environmentVariable = Environment.GetEnvironmentVariable("WLRegGetStatus");
153             WinLicenseSDK.GetEnvironmentVariable("WLRegGetStatus", stringBuilder, 100);
154             Logger.Main.LogString("WLRegGetStatus", stringBuilder.ToString());
155             Logger.Main.LogString("WLRegGetStatus Managed", environmentVariable);
156             int result;
157             if (string.IsNullOrEmpty(environmentVariable))
158             {
159                 result = -100;
160                 Logger.Main.LogError("ERROR retrieving WLRegGetStatus");
161             }
```

Esa flechita amarilla indica que estamos parados allí, justo en la línea 155 del código. Estamos viendo las líneas de código escritas por el/los programador/es (habitualmente son un equipo), es como tener los fuentes del programa, el poder es BRUTAL !

Analicemos esto un poco, tenemos la creación de un nuevo Objeto tipo StringBuilder (recordemos el operador *new* se utiliza para crear un objeto). Estamos en un mundo en que TODO son Objetos. Después se crea una variable tipo String a la que se le asigna el resultado de la llamada a GetEnvironmentVariable:

```
string environmentVariable = Environment.GetEnvironmentVariable("WLRegGetStatus");
```

El nombre de esa variable WLRegGetStatus nos suena y mucho de seguirle el rastro en el Olly y que terminaba en una llamada a la función nativa GetEnvironmentVariableW, recordemos:

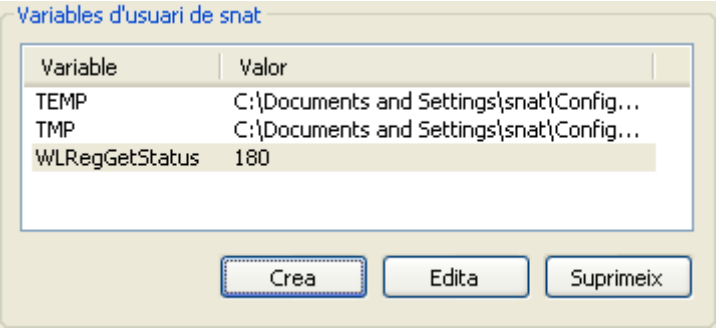
```
0012F2D4  03E90164  CALL to GetEnvironmentVariableW from 03E90162
0012F2D8  0140196C  VarName = "WLRegGetStatus"
0012F2DC  001A9D68  Buffer = 001A9D68
0012F2E0  00000080  BufSize = 80 (128.)
0012F2E4  79F010D8  mscorwks.79F010D8
0012F2E8  FFFFFFFF
0012F2EC  0012F33C
0012F2F0  79E808B0  RETURN to mscorwks.79E808B0 from ntdll.RtlSetLa
0012F2F4  79E808B8  RETURN to mscorwks.79E808B8 from mscorwks.79E71
```

Pasamos la instrucción actual con <F10> y entramos en la siguiente con <F11> (línea 152) y vemos esto:

```
Environment @020000AF
220         ///   <IPermission class="System.Security.Permissions.EnvironmentPermission, mscorlib, Vers
221         /// </PermissionSet>
222         // Token: 0x06000A72 RID: 2674 RVA: 0x0001F8F4 File Offset: 0x0001E8F4
223         public static string GetEnvironmentVariable(string variable)
224         {
225             if (variable == null)
226             {
227                 throw new ArgumentNullException("variable");
228             }
229             new EnvironmentPermission(EnvironmentPermissionAccess.Read, variable).Demand();
230             StringBuilder stringBuilder = new StringBuilder(128);
231             int i = Win32Native.GetEnvironmentVariable(variable, stringBuilder, stringBuilder.Capac
232             if (i == 0 && Marshal.GetLastWin32Error() == 203)
233             {
234                 return null;
235             }
```

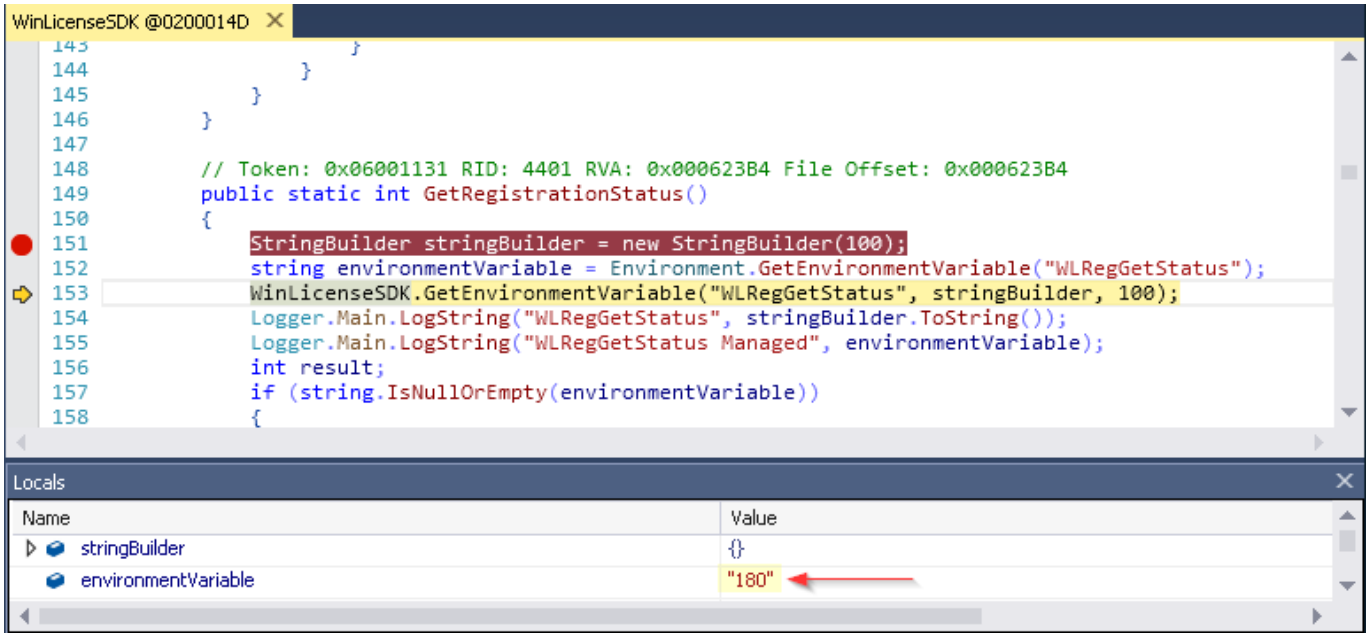
Se ve claro que llamará a Win32Native.GetEnvironmentVariable, que terminará en el kernel32 como ya vimos con Olly. Hagamos una primera prueba, crearemos la variable de entorno.

Le daremos un valor fácilmente reconocible como este:

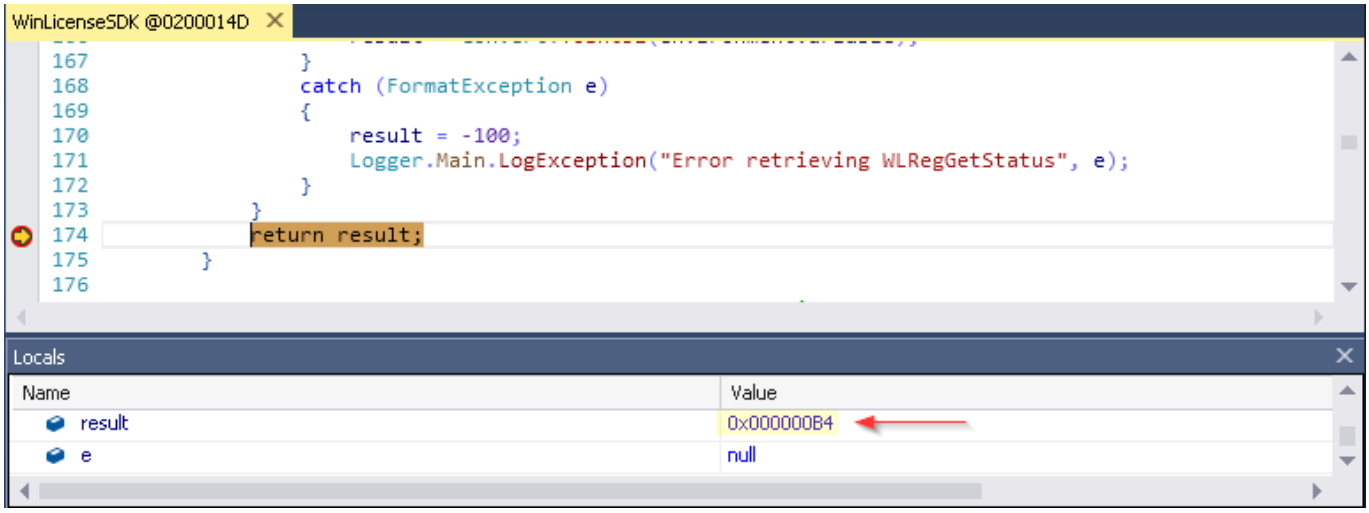


Cerramos el Debugger y lo volvemos a abrir, he tratado de sólo reiniciar el proceso cómo hacíamos con Olly pero no me ha funcionado.

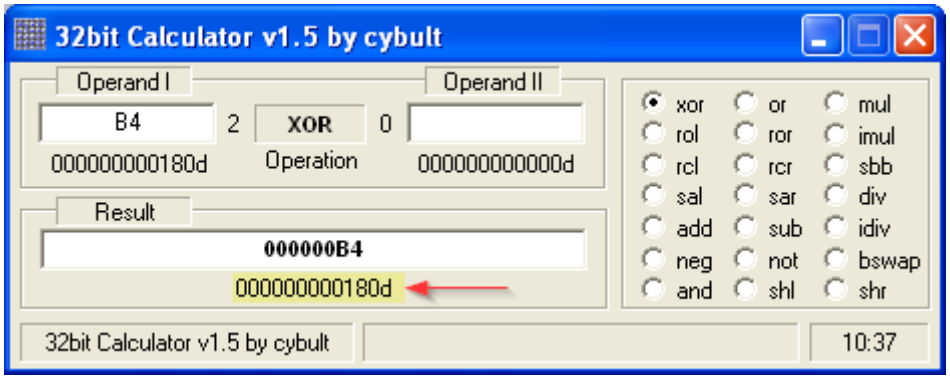
Llegamos al punto anterior (línea 231) donde presumiblemente leerá la variable, pasamos la instrucción y miramos la ventana Locals (donde están las variables):



Perfecto ! En la siguiente instrucción pasa el valor al StringBuilder que creó quedando igual a {180, sigamos traceando con <F10> hasta el RET:

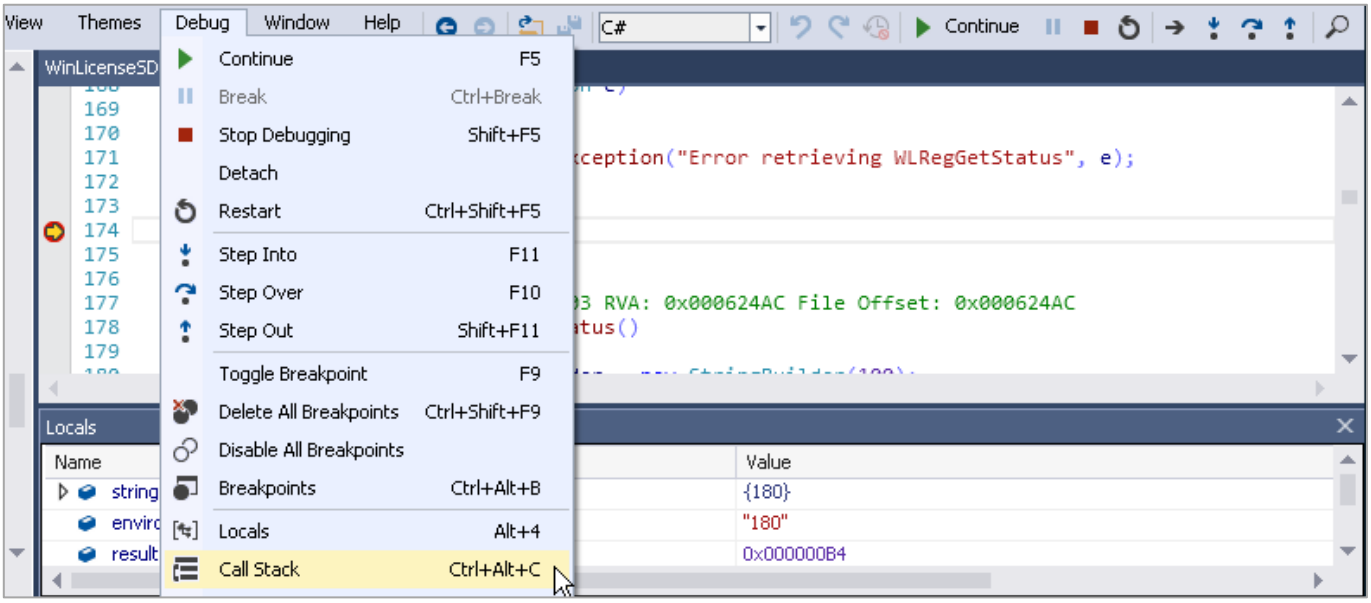


La función (el método en este caso) retornará 0x000000B4h (180 decimal) como podemos comprobar:

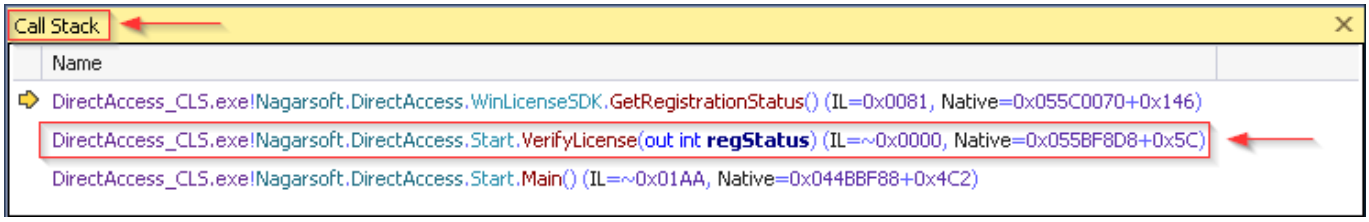


Nuestra calculadora de cybult nunca miente: 180 decimal, nuestra variable de entorno.

Veamos dónde retornará, hagamos un cambio en la ventana auxiliar (la inferior Locals), vamos a revisar el Call Stack (o pila de llamadas), cambiemos el modo vista:

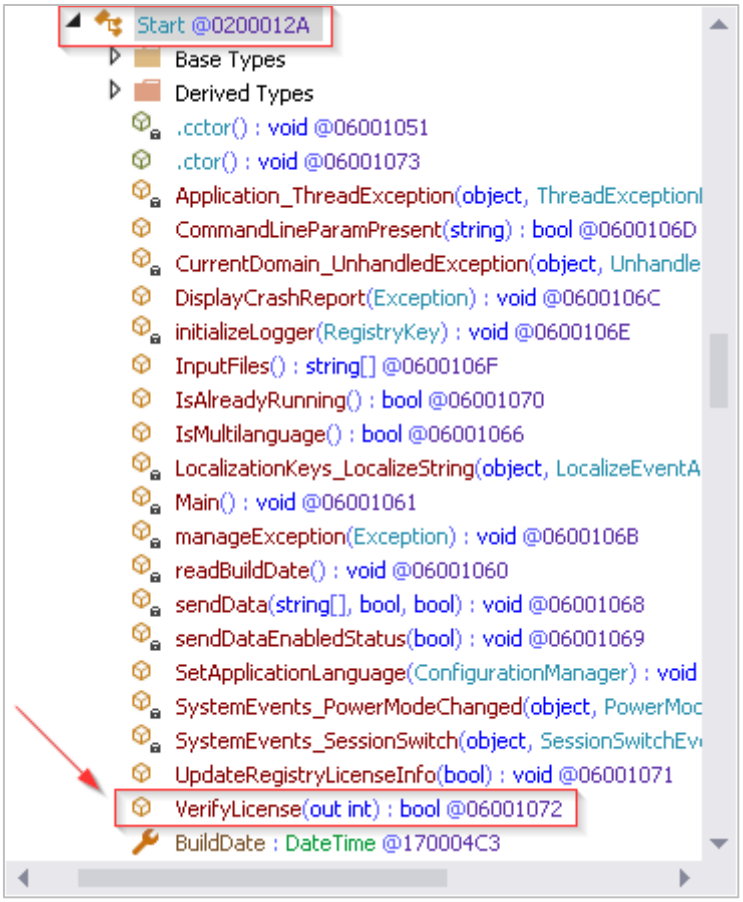


Esto nos deja el Call Stack visible en la ventana auxiliar (ahora Call Stack) y nos revela importante información hasta ahora desconocida por nosotros, mirad:



Resulta que el método GetRegistrationStatus() es invocado desde otro método: VerifyLicense() y que no vimos con .NET Reflector, aunque por su puesto también estaba allí.

Lo podemos encontrar en Nagarsoft.DirectAccess.Start.VerifyLicense(…) desplazándonos por el panel lateral izquierdo donde se listan los métodos con su contenido:



Esto comienza a mostrar la potencia de este debugger, lo que en Olly por ejemplo tardaríamos minutos por no decir horas de tedio traceo para encontrar este método, aquí se tardan unos dos minutos como mucho, además de contar con la posibilidad de trabajar con código en claro (y no con IL ni nada similar), por lo que esta potencia se multiplica * 3.

Contentos, muy contentos de ello seguimos nuestro camino:

Esta súper-NAG es lo que se nos ofrece. Como NO nos interesa la cerramos, estamos aquí:

```
Start @0200012A
579                 Logger.Main.LogInt("Invalid status code or license invalid for
580             }
581             frmRegistration frmRegistration = new frmRegistration();
582             DialogResult dialogResult = frmRegistration.ShowDialog();
583             if (dialogResult == DialogResult.Cancel)
584             {
585                 return false;
586             }
```

Ese frmRegistration.ShowDialog() nos sacó la ventana.

En la ventana Locals encontramos la respuesta que le dimos a la nag:

Locals	
Name	Value
frmRegistration	{Nagarsoft.DirectAccess.frmRegistration, Text: Welcome
dialogResult	Cancel
num	0x000000B4

Y retornaremos por el return false que nos cerrará la aplicación.

No se lo permitiremos, nos situamos sobre el Cancel (nuestra respuesta) y pulsamos <F2> para editar el valor, pero mejor será NO poner cualquiera, intentaremos fijar uno válido, cómo ? La ventana Locals tiene la respuesta que buscamos:

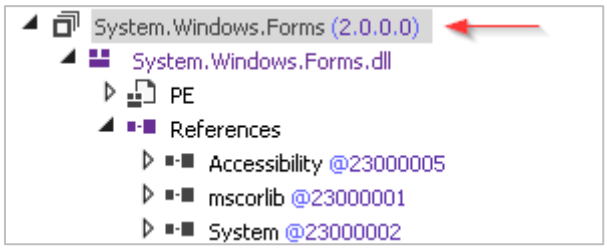
Locals	
	Type
	Nagarsoft.DirectAccess.frmUpgradeProtection/*0x02000045*/
ect Access 2.8....	Nagarsoft.DirectAccess.frmRegistration/*0x020000CF*/
	System.Windows.Forms.DialogResult/*0x01000120*/ {System.Windows.Forms.DialogResult/*0x020003B3*/}

Vayamos a Windows.Forms.DialogResult desde al panel lateral izquierdo:

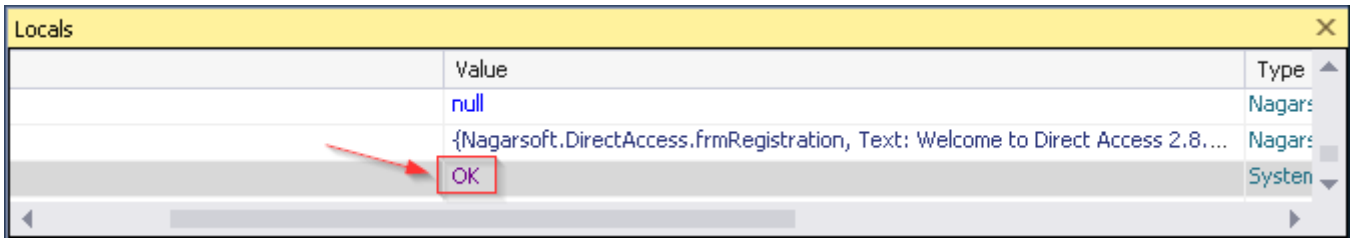
```
DialogResult @020003B3
    Base Types
    Derived Types
    Abort : DialogResult @04001CDB
    Cancel : DialogResult @04001CDA
    Ignore : DialogResult @04001CDD
    No : DialogResult @04001CDF
    None : DialogResult @04001CD8
    OK : DialogResult @04001CD9
    Retry : DialogResult @04001CDC
    value__ : int @04001CD7
    Yes : DialogResult @04001CDE
```

Allí están TODOS los valores de esta enumeración.

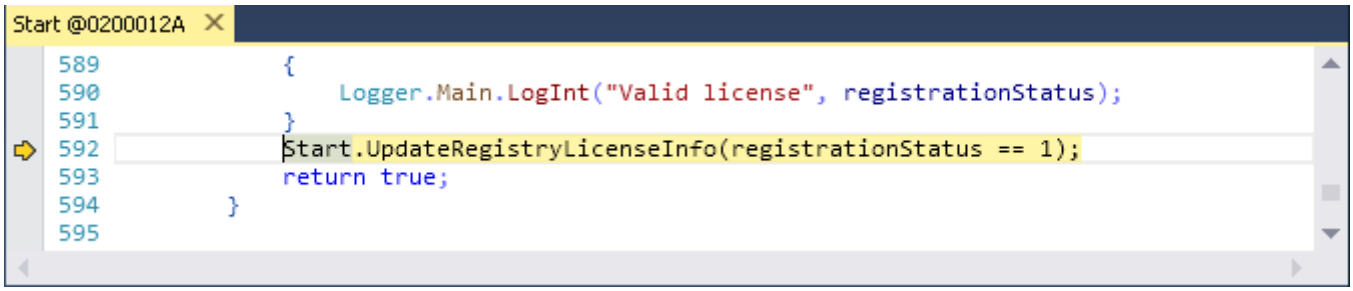
Estamos dentro de este módulo:



Hagamos el cambio Cancel por OK a ver qué pasa (yo todavía no lo he probado tampoco):



Parece que la cosa funciona porque la siguiente instrucción a ejecutar será esta:



Va a salir por el return true que nos interesa, además actualiza la información de Licencia con un poco de trampa porque el parámetro que se le pasa es este:

```
registrationStatus == 1
```

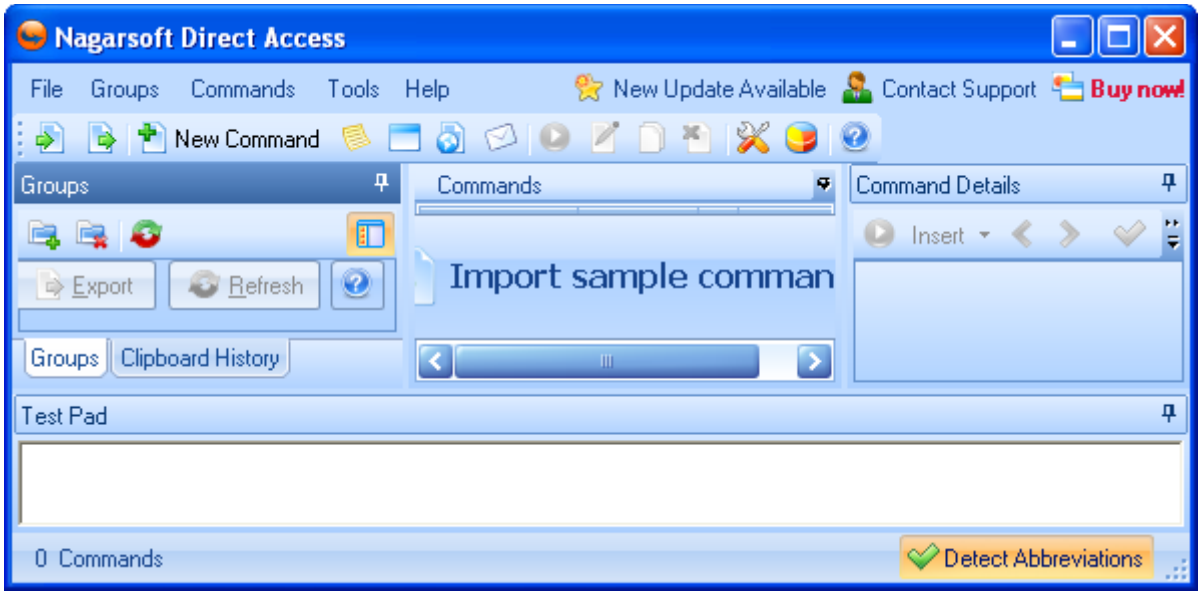
Con él hace lo siguiente:

1. Compara registrationStatus con 1
2. Si ambos SI valores son iguales retorna 1 >> Parámetro <- 1
3. Si ambos NO valores son iguales retorna 0 >> Parámetro <- 0

Por lo que el valor del parámetro vendrá determinado por el valor de la variable registrationStatus que ahora está a 0x00000B8h (180d), luego, el parámetro será 0x00h (salimos por el caso 2: ambos valores NO son iguales). Esta manera es utilizada en lenguajes de alto nivel en que las instrucciones por sentencia no están limitadas a una por línea (tal como pasa en ASM por ejemplo).

Aclarado esto, prosigamos la marcha..

Si dejamos correr la aplicación con <F5> el programa arranca bien:



Es una manera FORZADA de hacerlo, de ahí que en la barra de estado no ponga ni UNREGISTERED ni LICENSED, sencillamente no pone nada.

Si vamos a *Help -> About* el debugger se detiene en nuestro BP en la función GetRegistrationStatus(), (dicho queda que los métodos son funciones por lo que los nombraremos indistintamente de estas dos maneras) seguimos traceando con <F10> (equivalente al *Step Out* de Olly) y llegamos a este otro método:

```
frmMain @020000BF
4436
4437        // Token: 0x060009DE RID: 2526 RVA: 0x00041CB8 File Offset: 0x00041CB8
4438        private void showAboutForm()
4439        {
4440            int registrationStatus = WinLicenseSDK.GetRegistrationStatus();
4441            if (registrationStatus == 1 || registrationStatus == 12)
4442            {
4443                frmAbout frmAbout = new frmAbout();
4444                frmAbout.ShowDialog(this);
4445                frmAbout.Close();
4446                return;
4447            }
4448            frmRegistration frmRegistration = new frmRegistration();
4449            frmRegistration.ShowDialog(this);
4450            frmRegistration.Close();
4451        }
```

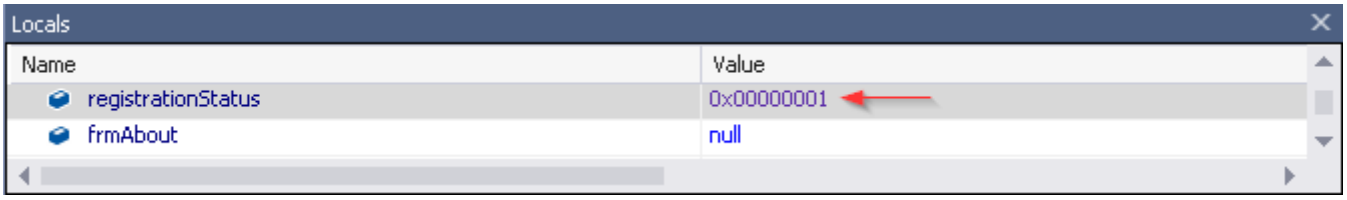
Donde se muestra la ventanita de About, es muy interesante ver que valora los valores 0x01h y 0xCh (12d) para la variable registrationStatus, si recordamos bien ya vimos esto en Olly en otra función, donde precisamente revisaba 0x01h y 0x0Ch y que eran valores supuestamente válidos para la variable de entorno WLRegGetStatus, podemos consultar el valor de la variable registrationStatus en la ventana Locals:

Locals

Name	Value
this	{Nagarsoft.DirectAccess.frmMain, Text: Nagarsoft Direct Acce
registrationStatus	0x000000B4

Vale 0x000000B0h (180d) Como nuestra variable de entorno.

Por lo que se NO entra en el IF (no se cumple la condición) y no saca el About MALO en la línea 4449, si editamos el valor de la variable registrationStatus a 0x01h o 0x0Ch, pongamos la a 0x01h:

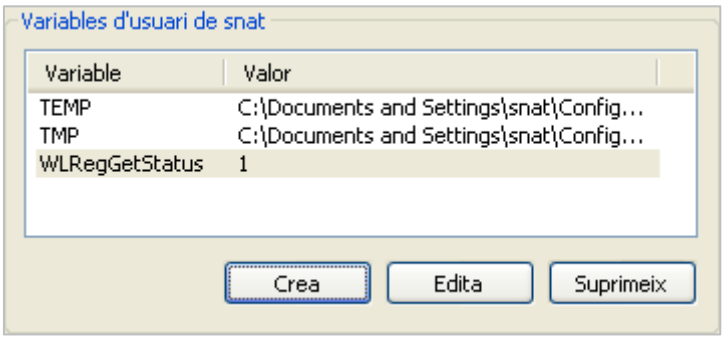


Si traceamos ahora con <F10> sí que entramos en el cuerpo del IF y se muestra otra About distinta:

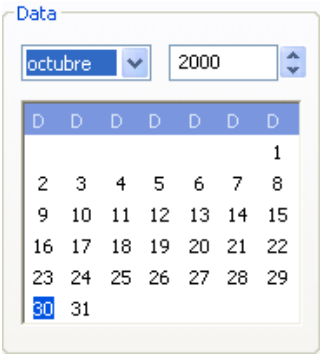
Esto es otra cosa.

Hasta este momento sabemos que manipulando los valores de las variables podemos desviar el flujo de ejecución tal como lo hacemos en Olly para ejecutables *no interpretados* (en C/C++, Delphi, etc..).

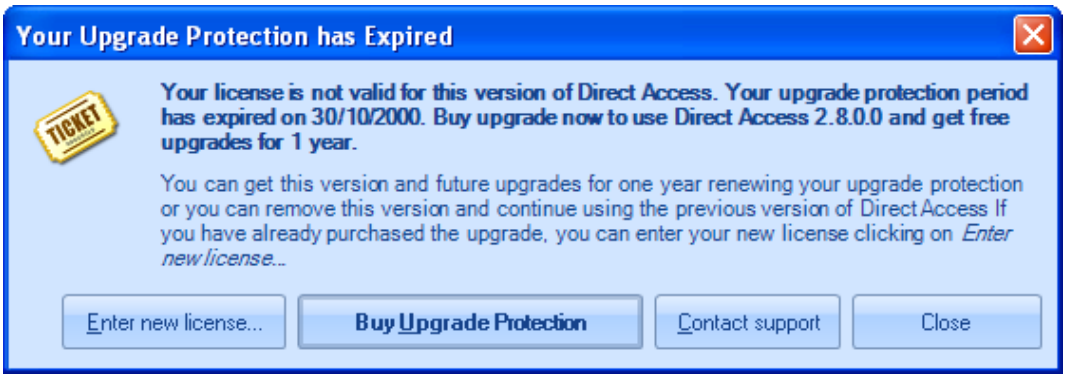
Vamos a ir un paso más, cambiemos TEMPORALMENTE la variable de entorno a 1:



Atrasemos el reloj hasta el año 2000:



Cerramos y abrimos dnSpy, ejecutamos el programa de nuevo y aparece esto:



Ya conocemos esta nag. Coloquemos un BP Start.VerifyLicense() y damos RUN con <F5> para parar allí:

```
Start @0200012A
474         // Token: 0x06001072 RID: 4210 RVA: 0x00061220 File Offset: 0x00061220
475         public static bool VerifyLicense(out int regStatus)
476         {
477             int registrationStatus = WinLicenseSDK.GetRegistrationStatus();
478             if (registrationStatus == -100)
479             {
480                 MessageBox.Show(string.Format(Resources.LicenseStatusExplanation, App
481                 regStatus = registrationStatus;
482                 return false;
483             }
484             bool flag = true;
485             regStatus = registrationStatus;
486             Logger.Main.LogInt("Registration Status", registrationStatus);
487             int num = registrationStatus;
488             switch (num)
489             {
490             case 0:
491             case 2:
492                 Start.EvaluationDaysLeft = WinLicenseSDK.EvaluationDaysLeft();
493                 if (registrationStatus == 0)
494                 {
495                     Logger.Main.LogMessage("License Status: wlIsTrial");
496                 }
497                 else if (registrationStatus == 2)
498                 {
499                     Logger.Main.LogMessage("License Status: wlInvalidLicense");
500                 }
501                 Start.RegistrationName = "<strong>" + Resources.UNREGISTEREDTRIAL + "
502                 goto IL_235;
503             case 1:
504                 break;
505             case 3:
506             case 4:
507             case 6:
508                 goto IL_235;
509             case 5:
510                 Logger.Main.LogMessage("License Status: wlLicenseExpired");
511                 goto IL_235;
512             case 7:
513                 Logger.Main.LogMessage("License Status: wlLicenseStolen");
514                 goto IL_235;
515             default:
516                 if (num != 12)
517                 {
518                     goto IL_235;
519                 }
520                 break;
521             }
```

Interesantísimo !! Si recordáis ya hablamos de un hipotético switch-case traceando con OllyDbg a la vista de esta función, cuyo código se veía así:

```
Address   Hex dump     Disassembly                          Comment
00F8F394  FF15 98789800 call  dword ptr [0x987898]
00F8F39A  83FE 08       cmp   esi, 0x8                     // ** WLRegGetStatus
00F8F39D  73 07         jnb   short 00F8F3A6
00F8F39F  FF24B5 38F7F80 jmp  dword ptr [esi*4+0xF8F738]
00F8F3A6  83FE 0C       cmp   esi, 0xC                     // ** ESI > 0x08h
00F8F3A9  74 6C         je    short 00F8F417
00F8F3AB  E9 F9020000   jmp   00F8F6A9
00F8F3B0  FF15 0C77E503 call  dword ptr [0x3E5770C]        // ** ESI==0x00h, 0x02h
00F8F3B6  A3 603C9800   mov   dword ptr [0x983C60], eax
00F8F3BB  85F6          test  esi, esi
00F8F3BD  75 15         jnz   short 00F8F3D4
00F8F3BF  8B0D 00742F02 mov   ecx, dword ptr [0x22F7400]
```

Ese salto mortal es el switch (num) de la captura anterior (tomada con Olly del tute del .NET anterior), allí se listaron TODOS los posibles casos, como se puede observar estábamos en lo cierto, veamos otra imagen:

```
Address   Hex dump     Disassembly                          Comment
00F8F6A9  837D D4 00    cmp   dword ptr [ebp-0x2C], 0x0    // ** ESI==0x03h, 0x04h, 0x06h
00F8F6AD  74 51         je    short 00F8F700               // ** ESI > 0x08h
00F8F6AF  85F6          test  esi, esi
00F8F6B1  75 15         jnz   short 00F8F6C8
00F8F6B3  8B0D 00742F02 mov   ecx, dword ptr [0x22F7400]
00F8F6B9  8B15 44A62F02 mov   edx, dword ptr [0x22FA644]
00F8F6BF  3909          cmp   dword ptr [ecx], ecx
00F8F6C1  E8 E22AFFFF   call  00F821A8
00F8F6C6  EB 15         jmp   short 00F8F6DD
00F8F6C8  56            push  esi
00F8F6C9  8B0D 00742F02 mov   ecx, dword ptr [0x22F7400]
00F8F6CF  8B15 48A62F02 mov   edx, dword ptr [0x22FA648]
```

Son EXACTAMENTE las mismas cláusulas CASE que se ve en dnSpy, estábamos traceando el código asociado al método VerifyLicense, cosas del destino :)

Este método tiene mucho que ver en la validación de la licencia y que, por ende, nos está bloqueando el paso al ejecutable protegido. Si traceamos el método hasta la instrucción del switch (línea 488) veremos claramente la cláusula que ejecuta en función del valor que lee:

```
Start @0200012A
483             }
484             bool flag = true;
485             regStatus = registrationStatus;
486             Logger.Main.LogInt("Registration Status", registrationStatus);
487             int num = registrationStatus;
488             switch (num)
489             {
490             case 0:
491             case 2:
492                 Start.EvaluationDaysLeft = WinLicenseSDK.EvaluationDaysLeft();
493                 if (registrationStatus == 0)
494                 {
```

Locals

Name	Value
dialogResult	None
num	0x00000001

Tenemos num==0x00000001h así que estamos en el caso 1 que es un break:

```
Start @0200012A
502                 goto IL_235;
503             case 1:
504                 break;
505             case 3:
```

Sigue por aquí:

```
Start @0200012A
526         string text4;
527         DateTime dateTime;
528         bool flag2;
529         WinLicenseSDK.GetRegistrationInfo(registrationStatus, out arg, out text, out
530         Start._serialNumber = text2.Trim();
531         Start.CustomerEmail = text3;
532         Logger.Main.LogString("SerialNumber", text2);
```

Esta función también la conocemos, entramos en ella con <F11> y vemos qué ejecutará:

```
WinLicenseSDK @0200014D
83
84          // Token: 0x06001137 RID: 4407 RVA: 0x00062644 File Offset: 0x00062644
85          public static void GetRegistrationInfo(int status, out string name, out string
86          {
87              name = string.Empty;
88              company = string.Empty;
89              email = string.Empty;
90              numlic = string.Empty;
91              serialnumber = string.Empty;
92              subscribeExpireDate = DateTime.Today;
93              powerLicensePack = false;
94              string a = string.Empty;
95              string text = string.Empty;
96              if (status == 1 || status == 12)
97              {
98                  StringBuilder stringBuilder = new StringBuilder(500);
99                  WinLicenseSDK.GetEnvironmentVariable("WLRegGetLicenseInfo", stringBui
100                 Logger.Main.LogString("WLRegGetLicenseInfo", stringBuilder.ToString()
101                 string[] array = stringBuilder.ToString().Split(new char[]
102                 {
103                     ','
104                 });
105                 string s = string.Empty;
106                 if (array.Length >= 3)
107                 {
108                     name = array[0];
```

Esta es la encargada de leer la segunda variable de entorno WLRegGetLicenseInfo que completamos así en el texto anterior:

- snat,CLS,123456-000;snat@gmail.com;2500;30000810;PowerLicense

Este es el método responsable de obtener y analizar el contenido de la misma, traceamos hasta aquí:

```
Start @0200012A
558         }
559         if (Start.buildDate > dateTime)
560         {
561             Logger.Main.LogMessage("License invalid for this version of the softwa
562             Start.RegistrationName = "<strong>" + Resources.UNREGISTEREDTRIAL + "
563             Start.IsLicensed = false;
564             new frmUpgradeProtection
565             {
566                 ExpireDate = dateTime
567             }.ShowDialog();
```

Se está efectuando una comparación de la que depende el que muestre o no la nag EXPIRED, además de sacarnos del método con un más que molesto return false que no nos interesa NADA.

De esta zona, además, obtuvimos una referencia desde OllyDbg:

```
Address   Hex dump    Disassembly                          Comment
00F8F5D1  8B5D BC     mov     ebx, dword ptr [ebp-0x44]
00F8F5D4  81E3 FFFFFFF3 and   ebx, 0x3FFFFFFF
00F8F5DA  3BD3        cmp     edx, ebx
00F8F5DC  7C 06       jl      short 00F8F5E4
00F8F5DE  7F 08       jg      short 00F8F5E8
00F8F5E0  3BC1        cmp     eax, ecx
00F8F5E2  77 04       ja      short 00F8F5E8
00F8F5E4  33C0        xor     eax, eax
00F8F5E6  EB 05       jmp     short 00F8F5ED
00F8F5E8  B8 01000000 mov     eax, 0x1
Jump is taken

Registers (FPU)
ESI 0000000C
EDI 00000000
EIP 00F8F5DC
C 0  ES 0023 32b
P 1  CS 001B 32b
A 1  SS 0023 32b
Z 0  DS 0023 32b
S 0  FS 003B 32b
T 0  GS 0000 NUL
D 0
O 1  LastErr ERR
```

Además invertimos el salto JL para que el programa se ejecutara.

Veamos qué valor tiene la variable dateTime:

Locals	
Name	Value
dateTime	{30/10/2000 00:00:00}
flag	false
flag2	false

Nuestra fecha atrasada al año 2000. Desde la línea actual (línea 559) analicemos la variable Start.buildDate:

```
Start @0200012A
555                 frmM
556                 retu
557             }
558         }
559         if (Start.buildDate > dateTime)
560         {
```

Go to MD Table Row...	Ctrl+Shift+D
Analyze	Ctrl+R
Find	Ctrl+F

En el panel auxiliar se nos mostrará información de TODOS los lugares desde los que esta variable es accedida o modificada, expandimos el árbol *Assigned by* (asignado por):

```
Analyzer
Nagarsoft.DirectAccess.Start.buildDate : DateTime @04000AB2
  Read By
  Assigned By
    Nagarsoft.DirectAccess.Start..cctor() : void @06001051
    Nagarsoft.DirectAccess.Start.readBuildDate() : void @06001060
```

El primer punto es en el constructor de Clase .cctor() donde se le asigna la fecha de hoy:

```
Start.buildDate = DateTime.Today;
```

Además la variable buildDate es una variable de clase, tal como hemos estudiado antes, se puede reconocer por la notación utilizada:

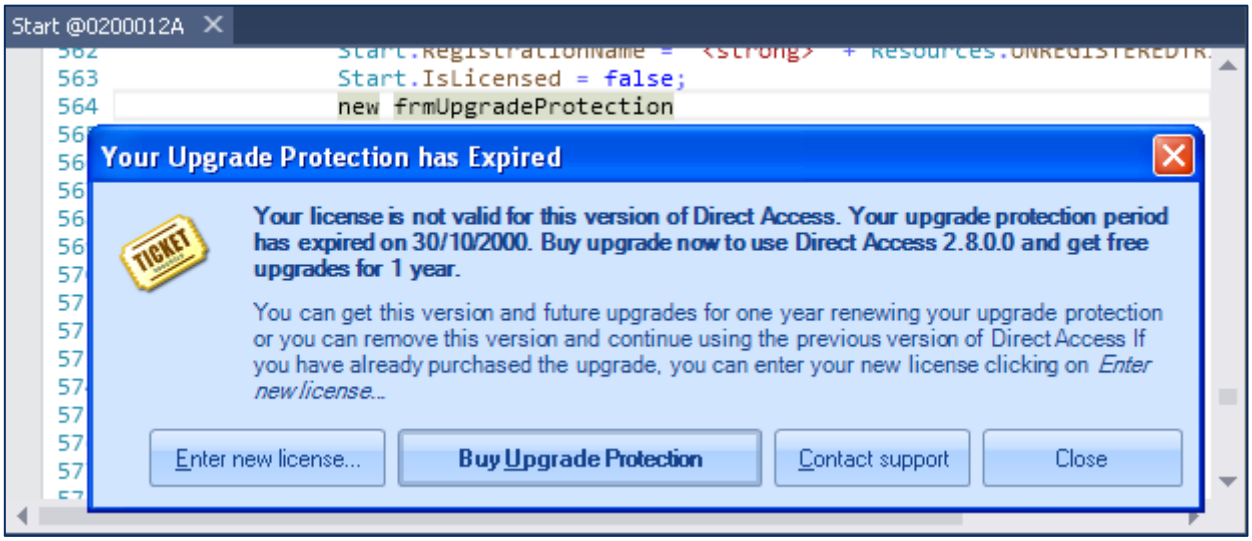
```
Clase. Campo
```

De momento nos dirigimos al método readBuildDate() a ver qué vemos:

```
readBuildDate() : void @06001060
1  // Nagarsoft.DirectAccess.Start
2  // Token: 0x06001060 RID: 4192 RVA: 0x000607D0 File Offset: 0x000607D0
3  private static void readBuildDate()
4  {
5      try
6      {
7          string creationInfo = Resources.CreationInfo;
8          string[] array = creationInfo.Split(new char[]
9          {
10             '-'
11         });
12         int year = Convert.ToInt32(array[0]);
13         int month = Convert.ToInt32(array[1]);
14         int day = Convert.ToInt32(array[2]);
15         Start.buildDate = new DateTime(year, month, day);
16     }
```

En la última línea de código justo antes de la llave de cierre se ve la asignación y poco más podemos sacar de aquí, si seguimos traceando nos saca la nag:

```
Start @0200012A
563                 Start.IsLicensed = false;
564                 new frmUpgradeProtection
```



Y nos echa fuera con return false:

```
Start @0200012A
561                 Logger.Main.LogMessage("License invalid for this version of th
562                 Start.RegistrationName = "<strong>" + Resources.UNREGISTEREDTR
563                 Start.IsLicensed = false;
564                 new frmUpgradeProtection
565                 {
566                     ExpireDate = dateTime
567                 }.ShowDialog();
568                 return false;
569             }
```

Ya sabemos de qué va la copla.

Posteriormente nos saca de la rutina directamente:

```
Start @0200012A
325                 Start.DisplayCrashReport(ex);
326             }
327             finally
328             {
329                 SystemEvents.SessionSwitch -= new SessionSwitchEventHandler(St
330                 Logger.Main.LogMessage("Unregistering Locking/Unlocking notifi
331                 SystemEvents.PowerModeChanged -= new PowerModeChangedEventHand
332                 Logger.Main.LogMessage("Unregistering Suspension/Resume notifi
333                 Logger.Main.LeaveProcess(Application.ProductName);
334             }
335         }
```

Salimos por el finally bloque, éste se utiliza para el tratamiento de excepciones, colocando el código vigilado en una cláusula como esta:

```
try {
            // <Código vigilado>
            // <Sólo se ejecuta si NO hay excepción>
} catch   {
            // <Se genera una excepción, continúa ejecución desde aquí..>
} finally {
            // <Siempre se ejecuta este código. ocurra o NO ocurra una excepción>
}
```

Se estructura en bloques como los aquí mostrados, se ejecuta un código susceptible de generar una excepción, para capturarla y manejarla se establece la cláusula catch donde se ejecutarán las sentencias que sean necesarias SI la excepción es levantada (raised), en caso de no suceder continuará la normal ejecución en la línea siguiente del bloque try, y en TODOS los casos se ejecutará el código del bloque finally.

En la línea 335 termina el programa, nos ha echado fuera.

Lleguemos otra vez a la línea 559 y haremos un cambio, de esto:

Locals

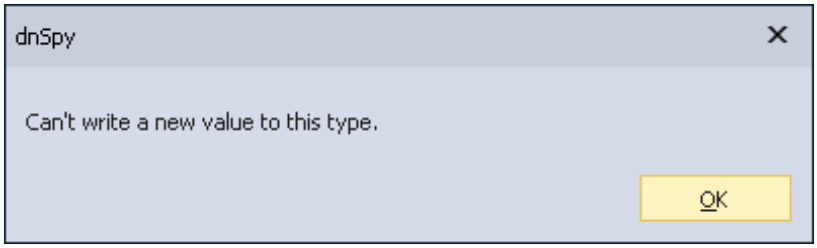
Name	Value	Type
dateTime	{30/10/2000 00:00:00}	System.DateTime/*0x01000070*/
flag	false	bool

A esto:

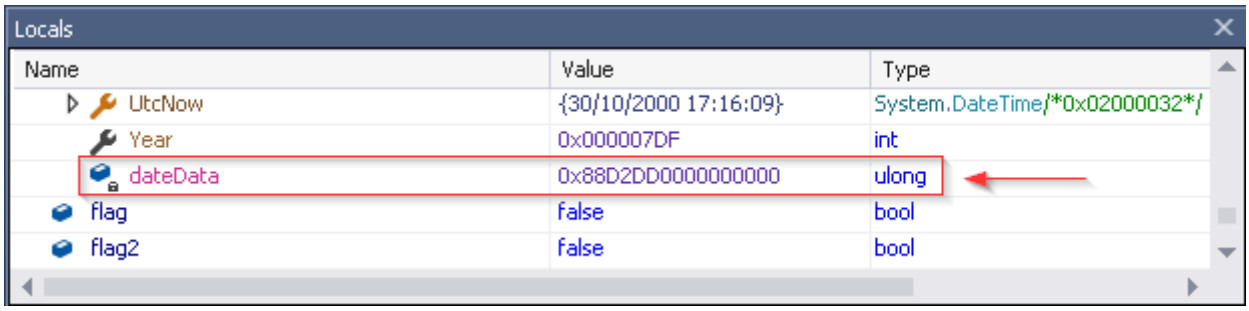
Locals

Name	Value	Type
text2	""	string
dateTime	{25/10/2015 05:49:08}	System.DateTime/*0x01000070*/
flag	false	bool
flag2	false	bool
netInstances	0x00000000	int
frmMoreLicensesRequired	null	Nagarsoft.DirectAccess.frmMoreLic
V_11	null	Nagarsoft.DirectAccess.frmUpgrad

Esto se logra modificando el valor del Objeto dateTime (tipo System.Date.Time), pero si intentamos hacerlo a lo bruto, con <F2> sobre él para editarlo pasará esto:

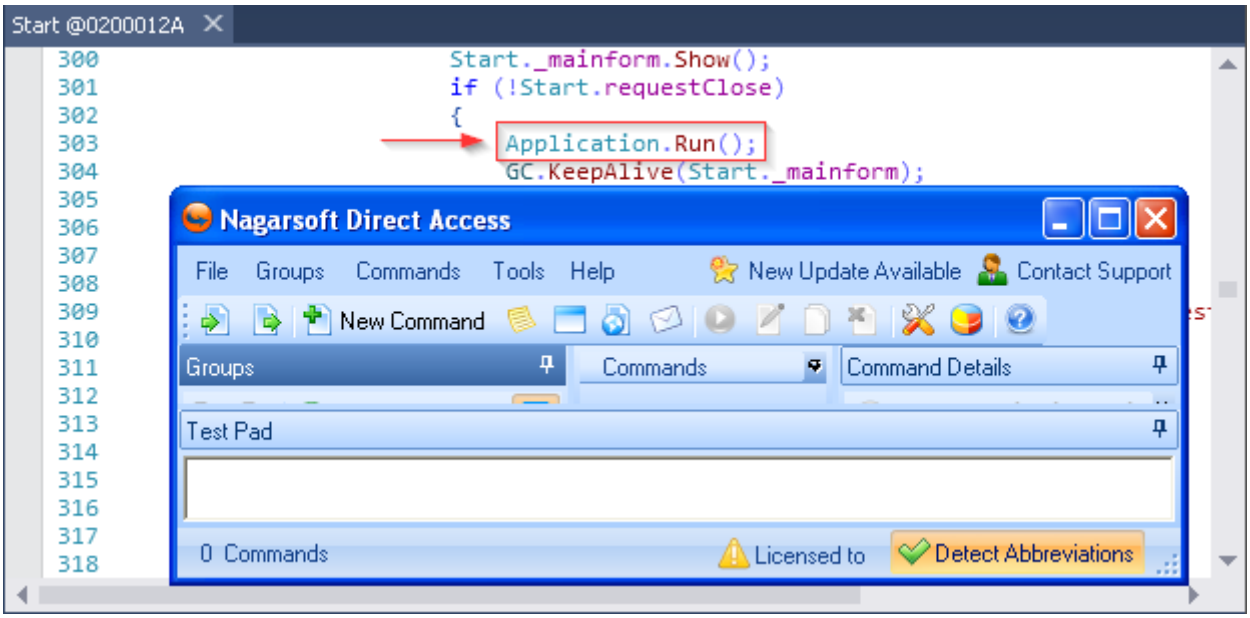


Lo que hay que hacer es EXPANDIR el objeto y modificar su contenido de esta manera:



Ahí está nuestra fecha (tipo unsigned long, o entero largo sin signo), Ahora sí, sigamos traceando con <F10> y llegaremos a la que parece ser la zona buena:

Si continuamos con <F10> en seguida aparece la ventana principal del programa en pantalla:



Y allí queda *flotando* en el escritorio :)

Recapitulemos, para que el programa se ejecute necesitamos:

1.- que el método GetRegistrationStatus() retorne siempre 1:
2.- Evitar caer en el IF de la línea 559.

Esto es lo mismo decir que el método Start.VerifyLicense() retorne SIEMPRE true.

Y con esto lograríamos, además, que el ejecutable corra indistintamente de la fecha actual.

Esto que parece tan sencillo de hacer no lo es tanto teniendo en cuenta que el código que hay que modificar es código IL (o Código Intermedio).

Lo que nos lleva, inexorablemente, a la siguiente sección..

5. Código IL ?

Pues sí, esto es, código IL justo lo que necesitamos para poder modificar el assembly. Para tener una visión más cercana veamos una comparativa de código nativo con código IL (CIL o MSIL):

- código nativo x86

```
add eax, edx
```

- código CIL:

```
ldloc.0
ldloc.1
add
stloc.0     // a = a + b or a += b;
```

Extraído de Wikipedia:

- ✓ https://en.wikipedia.org/wiki/Common_Intermediate_Language

Ya lo vemos, nada que ver con lo que nosotros conocemos, entonces antes de modificar código arbitrariamente debemos conocer cómo funcionan los entresijos del lenguaje.

Hay varias etapas en el procesamiento del mismo, desde que escribimos código en C#/VB .NET hasta que se ejecuta en nuestra CPU. Este código escrito se traducirá a código CIL (también llamado IL o MSIL) en lugar de hacerlo a código objeto como lo hace por ejemplo un compilador de C. Este código CIL es independiente de la CPU y podrá, posteriormente, ser ejecutado mediante la Infraestructura de Lenguaje Común (CLI), no hay que confundir los términos CIL (Common Intermediate Language) con CLI (Common Language Infrastructure) que es el .NET Runtime, con esto se consigue un código multiplataforma (al estilo de java) sin necesidad de recompilar los fuentes.

El proceso general de tratamiento de los mismos sería el siguiente:

1. El código fuente se traduce a código CIL en una primera etapa.
2. Este código CIL se ensamblará en un bytecode y se creará el archivo assembly. (EXE o DLL)
3. Este assembly, en su ejecución se pasará al compilador JIT de turno, quién generará el código nativo (específico de nuestra plataforma).
4. Nuestra CPU ejecutará el código nativo generado por el compilador JIT.

Es obligatorio saber esto ? Por su puesto que no, pero para manipular con garantías el .NET sí que es bueno conocer lo máximo posible de éste, contra más información tengamos mejor podremos después modificarlo. Hemos visto como transforma un sencillo add eax, edx en un galimatías de instrucciones y si queremos modificar el flujo de ejecución de nuestro assembly estudiado (DirectAccess.exe) es una buena oportunidad de aprender este nivel del lenguaje aunque sea superficialmente.

Además de conocer el proceso de creación y ejecución del assembly, necesitamos saber el modo en que realiza las operaciones durante su ejecución. Investigando estos días he podido observar que utiliza un método de varias etapas por así decirlo.

Vamos a ver si me explico un poco. Imaginemos que queremos ejecutar una sencilla instrucción tal que así:

mov arg1, 3

Partimos del hecho que arg1 es un parámetro recibido por procedimiento (o método) en el que queremos aplicar la modificación (una variable normal y corriente).

El proceso de asignación se realizará en tres etapas:

1. cargará el parámetro en el stack.
2. cargará el valor entero 3 al stack.
3. extraerá el valor del stack hacia la variable arg1 (nuestro parámetro).

Lo primero que debemos hacer es hacernos con una tabla de Opcodes IL, se puede obtener una aquí:

https://msdn.microsoft.com/en-us/library/system.reflection.emit.opcode

Aunque supongo que habrá disponibles en otros lugares.

Mirando la tabla encontramos estas instrucciones:

S	Ldarg	Loads an argument (referenced by a specified index value) onto the stack.
S	Ldarg_0	Loads the argument at index 0 onto the evaluation stack.
S	Ldarg_1	Loads the argument at index 1 onto the evaluation stack.
S	Ldarg_2	Loads the argument at index 2 onto the evaluation stack.
S	Ldarg_3	Loads the argument at index 3 onto the evaluation stack.
S	Ldarg_S	Loads the argument (referenced by a specified short form index) onto the evaluation stack.
S	Ldarga	Load an argument address onto the evaluation stack.
S	Ldarga_S	Load an argument address, in short form, onto the evaluation stack.

Estas se utilizan para pasar argumentos al stack, partiendo de Ldarg y sus variantes.

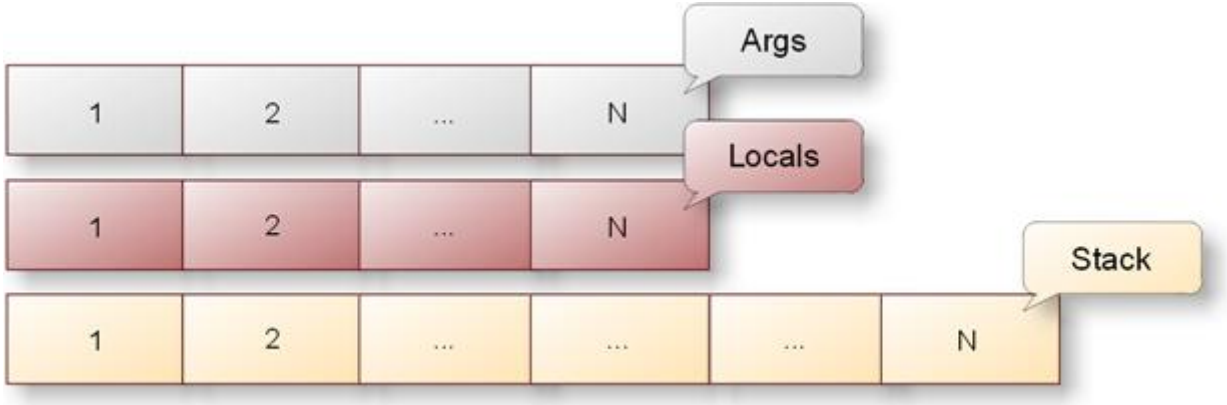
Hay otras para pasar variables locales en lugar de parámetros como estas:

S	Ldloc	Loads the local variable at a specific index onto the evaluation stack.
S	Ldloc_0	Loads the local variable at index 0 onto the evaluation stack.
S	Ldloc_1	Loads the local variable at index 1 onto the evaluation stack.
S	Ldloc_2	Loads the local variable at index 2 onto the evaluation stack.
S	Ldloc_3	Loads the local variable at index 3 onto the evaluation stack.
S	Ldloc_S	Loads the local variable at a specific index onto the evaluation stack, short form.
S	Ldloca	Loads the address of the local variable at a specific index onto the evaluation stack.
S	Ldloca_S	Loads the address of the local variable at a specific index onto the evaluation stack, short form.

Allí se ve la aclaración: *Loads the local variable at specific index onto the evaluation stack*

Con la instrucción Ldarg tendríamos para el punto 1 (cargar el parámetro en el stack). Nos quedaría buscar las otras dos y armar la instrucción. Hay que notar cómo se refiere al stack como *evaluation stack*, un stack trial de 30 días ? :DDDD

Con estas instrucciones podemos pasar tanto argumentos (parámetros del método actual) como variables locales al propio método al stack para después operar con ellos. Esto nos hace pensar en esta estructura:



Partimos de tres arrays:

1. Array Args: donde irán los argumentos para el método actual
2. Array Locals: donde se almacenarán las variables locales del método
3. Evaluation Stack: Donde se irán colocando y sacando variables locales, argumentos, campos estáticos, elementos de arrays y demás valores necesarios para operar.

Esto nos da información necesaria de las transiciones que sufrirá el valor que queramos establecer y/o modificar. Recordemos que partimos con un sencillo mov arg1, 3 como premisa.

La primera instrucción necesaria será empujar el valor 0x03h al stack, para ello todavía no hemos visto qué opcodes tenemos disponibles, hay éstos:

S	Ldc_I4	Pushes a supplied value of type **int32** onto the evaluation stack as an **int32**.
S	Ldc_I4_0	Pushes the integer value of 0 onto the evaluation stack as an **int32**.
S	Ldc_I4_1	Pushes the integer value of 1 onto the evaluation stack as an **int32**.
S	Ldc_I4_2	Pushes the integer value of 2 onto the evaluation stack as an **int32**.
S	Ldc_I4_3	Pushes the integer value of 3 onto the evaluation stack as an **int32**.
S	Ldc_I4_4	Pushes the integer value of 4 onto the evaluation stack as an **int32**.
S	Ldc_I4_5	Pushes the integer value of 5 onto the evaluation stack as an **int32**.
S	Ldc_I4_6	Pushes the integer value of 6 onto the evaluation stack as an **int32**.

El listado continúa hasta Ldc_I4_8 (equivalente a push 0x08h), Ldc_I4_M1 (equivalente a push -0x01h) y después Ldc_I4_S que es la forma corta del opcode (short form).

Nos quedamos con otras dos posibilidades

- ldc.14.3
- ldc.14 3

La primera es la forma directa y la segunda necesita, adicionalmente, el valor del entero a empujar al Evaluation Stack (le llamaremos stack).

La segunda instrucción que necesitamos será pasar el segundo parámetro (con índice 1 en el array de Args) al evaluation Stack. Si estudiamos bien la tabla de opcodes veremos que esto lo podemos lograr de varias maneras distintas, una sería esta:

- ldarg.1
- ldarg <parámetro>

La primera empuja el parámetro con índice 1 en el array de args al stack, la segunda es una variante donde se le debe decir, además, el índice del mismo, en este caso 1. Con cualquiera de estas tendríamos la variable en el stack lista para trabajar con ella.

Y nos queda la última que será asignar el valor entero 3 al parámetro en cuestión, como ambas ya están en el stack necesitamos una asignación, veamos la tabla a ver qué encontramos:

S	Stind_I	Stores a value of type **native int** at a supplied address.
S	Stind_I1	Stores a value of type **int8** at a supplied address.
S	Stind_I2	Stores a value of type **int16** at a supplied address.
S	Stind_I4	Stores a value of type **int32** at a supplied address.
S	Stind_I8	Stores a value of type **int64** at a supplied address.
S	Stind_R4	Stores a value of type **float32** at a supplied address.
S	Stind_R8	Stores a value of type **float64** at a supplied address.
S	Stind_Ref	Stores a object reference value at a supplied address.

Esta parece buena, hay muchas variantes, para int, para float, etc, además también hay para referencias.

En cualquier caso parece que tenemos el SET completo, nuestra instrucción quedará de la siguiente forma:

```
ldarg.1
ldc.14.3
stind.i4
```

Cómo comprobarlo ? Con nuestra súper herramienta dnSpy.

Cargamos DirectAccess_CLS.exe y llegamos al método GetRegistrationInfo():

```
WinLicenseSDK @0200014D
83
84    // Token: 0x06001137 RID: 4407 RVA: 0x00062644 File Offset: 0x00062644
85    public static void GetRegistrationInfo(int status, out string name, out str
86    {
87        name = string.Empty;
88        company = string.Empty;
89        email = string.Empty;
90        numlic = string.Empty;
91        serialnumber = string.Empty;
92        subscribeExpireDate = DateTime.Today;
93        powerLicensePack = false;
94        string a = string.Empty;
95        string text = string.Empty;
```

argumento2 (Args[2])

Trataremos de asignar el valor "3" al parámetro name, hay que editar las instrucciones IL del método:

```
WinLicenseSDK @0200014D
83
84    // Token: 0x06001137 RID: 4407 RVA: 0x00062644 File Offset: 0x00062644
85    public static void GetRegistrationInfo(int status, out string name, out str
86    {
87        name = string.Empty;
88        company = string.Empty
89        email = string.Empty;
90        numlic = string.Empty;
91        serialnumber = string.
92        subscribeExpireDate =
93        powerLicensePack = fal
94        string a = string.Empt
95        string text = string.E
```

Add Breakpoint	F9
Show Next Statement	Alt+Num *
Edit IL Instructions...	Ctrl+E
Go to MD Token...	Ctrl+D
Go to Entry Point	
Show Instructions in Hex Editor	Ctrl+X

Se abrirá una nueva ventanita donde vemos las instrucciones IL:

Edit Method Body - GetRegistrationInfo(int, out string, out string, out string, out string, out string,...

Instructions | Locals | Exception Handlers

Body Type: IL Code Type: IL

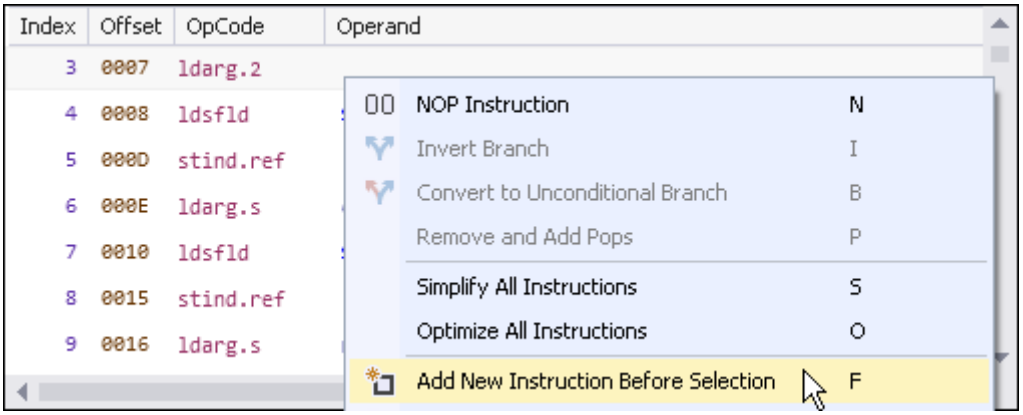
☐ Keep Old MaxStack ☑ Init Locals Header RVA Header Offset MaxStack LocalVarSigTok

Index	Offset	OpCode	Operand
0	0000	ldarg.1	
1	0001	ldsfld	string [mscorlib]System.String::Empty
2	0006	stind.ref	Instrucción 1
3	0007	ldarg.2	
4	0008	ldsfld	string [mscorlib]System.String::Empty

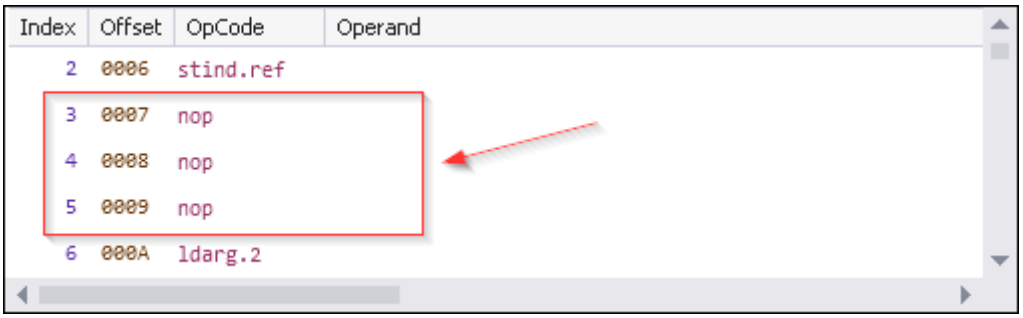
OK | Cancel | Restore

La línea lila denota la división entre la primera instrucción name = string.Empty que ocupa tres líneas con las siguientes instrucciones. La segunda comienza en el offset 0007, y así sucesivamente.

Lo que haremos será insertar instrucciones ANTES del offset 0007:

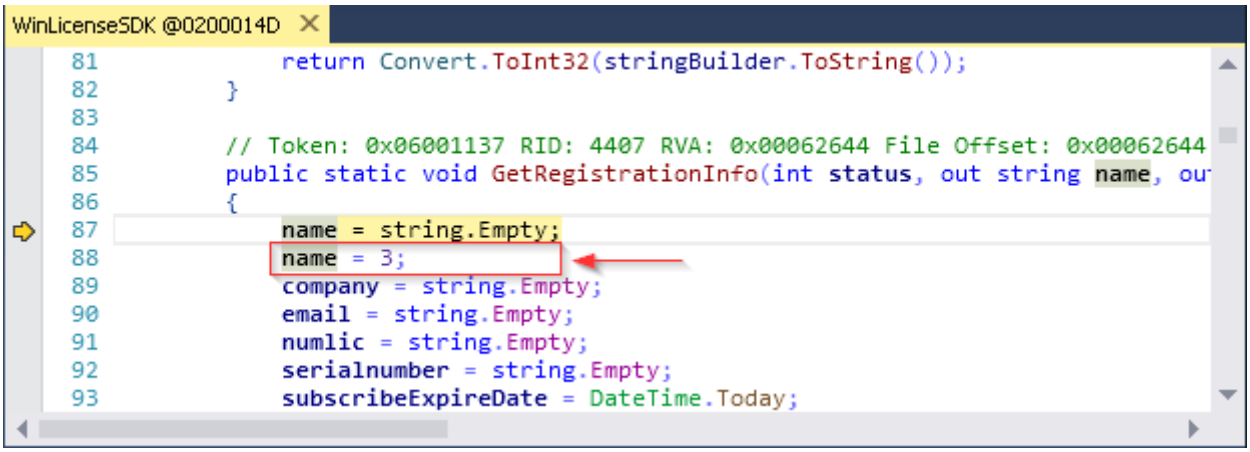


Repetimos la operación dos veces más, tres en total, nos debe quedar así:



A continuación las completamos con la información que tenemos:

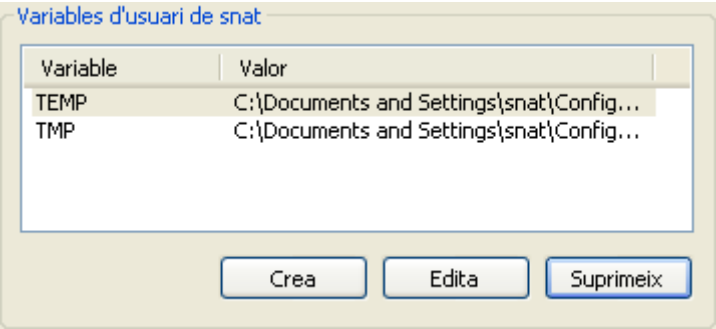
Pulsamos OK y veamos cómo quedó el código:



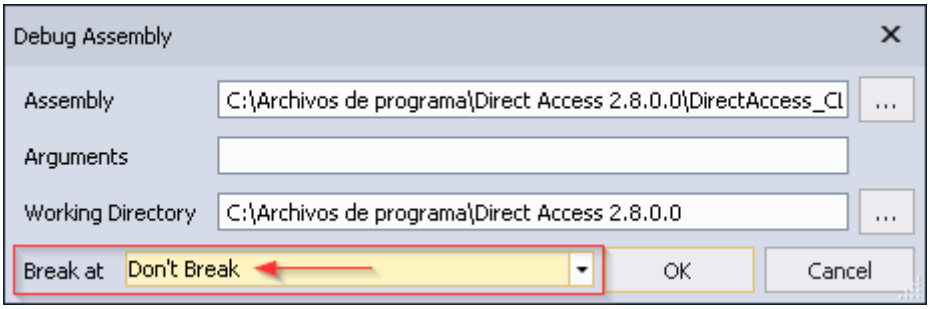
Genial, allí está nuestra primera instrucción :DD

Pero no nos emocionemos mucho que todavía queda mucho trabajo por hacer. Este es un principio. Además tampoco está del todo bien construida pues le asignamos un valor tipo entero a una variable tipo string. Eso sí, como prueba está fenomenal.

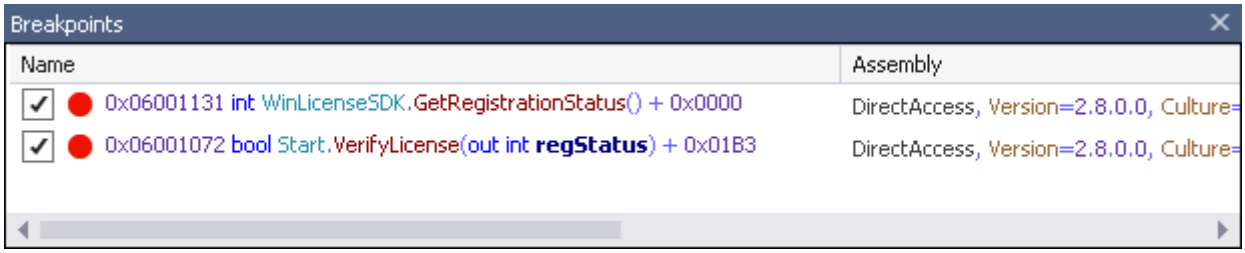
Eliminemos la variable de entorno WLRegGetStatus, no la utilizaremos más:



El siguiente paso será tratar de hacer alguna asignación con más sentido, tenemos dos puntos clave donde podemos modificar el código del ejecutable para que corra bien. La primera es el valor de retorno del método GetRegistrationStatus(), necesitamos que retorne 1. Lo haremos al vuelo, cargamos el assembly DirectAccess.exe en dnSpy, pulsamos Start <F5> y en la ventana que aparece lo configuramos para que no pare en el BreakPoint inicial:



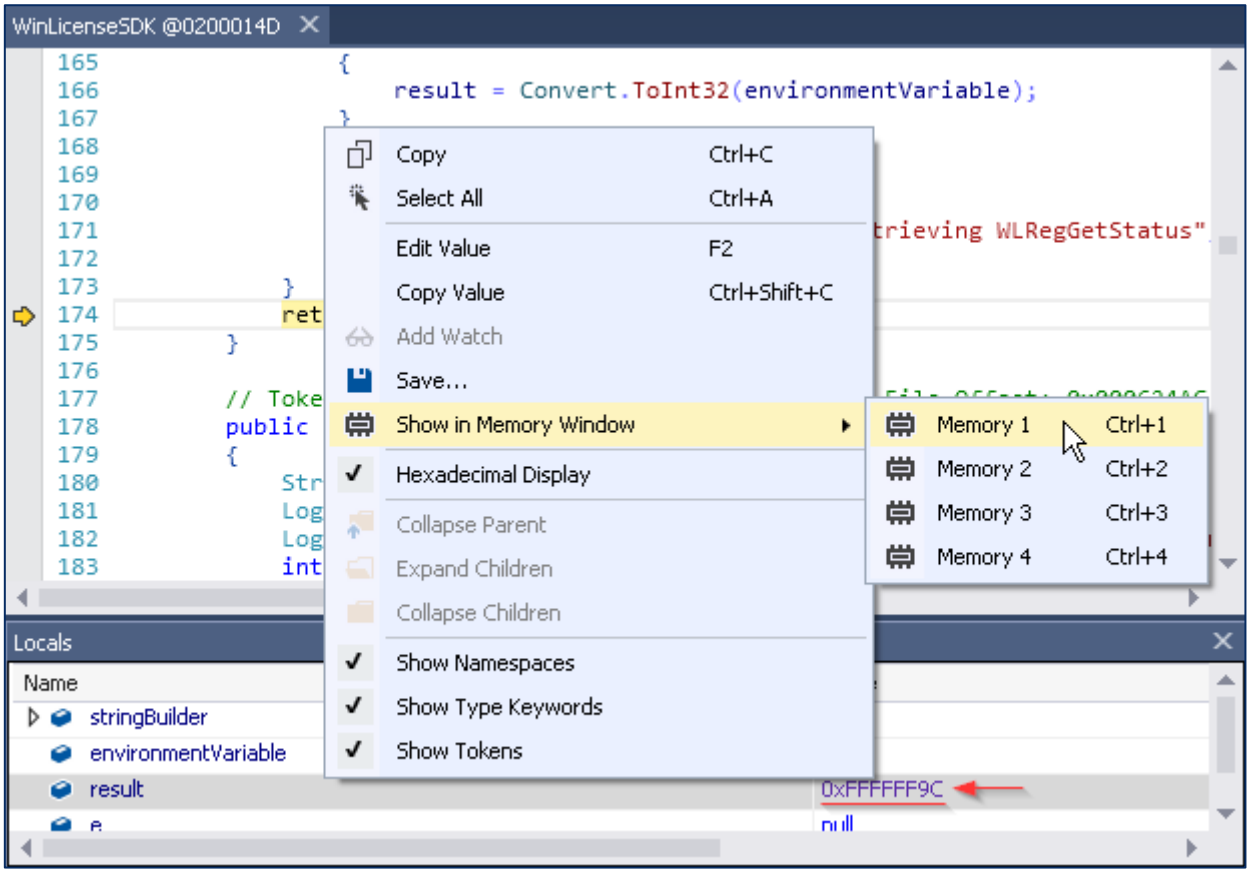
Sólo pararemos en los BP que pongamos nosotros, que podemos consultar en cualquier momento desde el menú *Debug -> Breakpoints* y que se mostrarán en la ventana auxiliar (panel inferior):



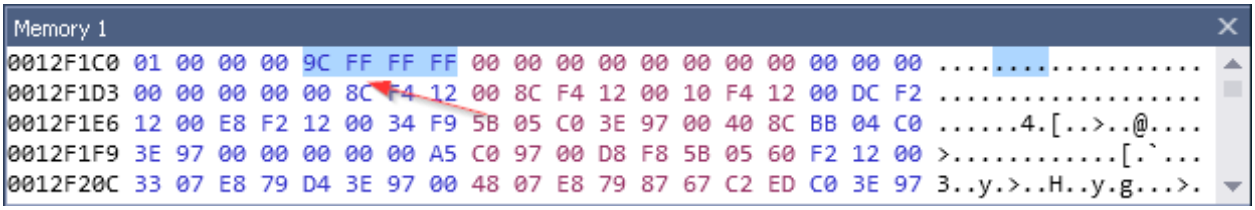
Tenemos estos dos: el del inicio del método GetRegistrationStatus() y el de la comparación entre las variables Start.buildDate y dateTime de la línea 599 y que controlan la aparición de la nag EXPIRED.

Puesto que estamos aquí para practicar con el debugger y que la limitación del programa es el medio para jugar con él vamos a modificar el valor de retorno al vuelo, llegaremos al RET de GetRegistrationStatus() y cambiaremos el valor de la variable result a 0x01h, pero desde el editor hexa incorporado del debugger.

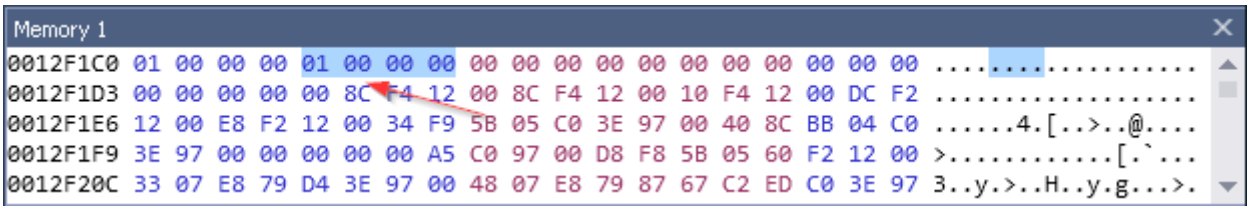
Para esto basta con pulsar con el botón derecho sobre la variable en la ventana Locals y elegir *Show in memory Window -> Memory 1* tal como se muestra en la siguiente imagen:



Y cambiamos esto:

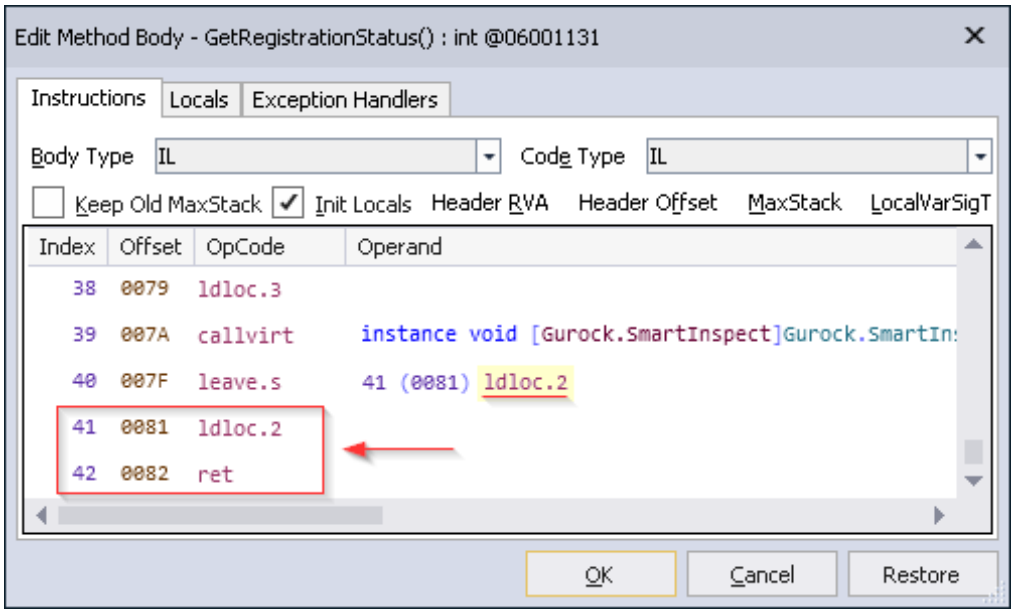


Por esto:

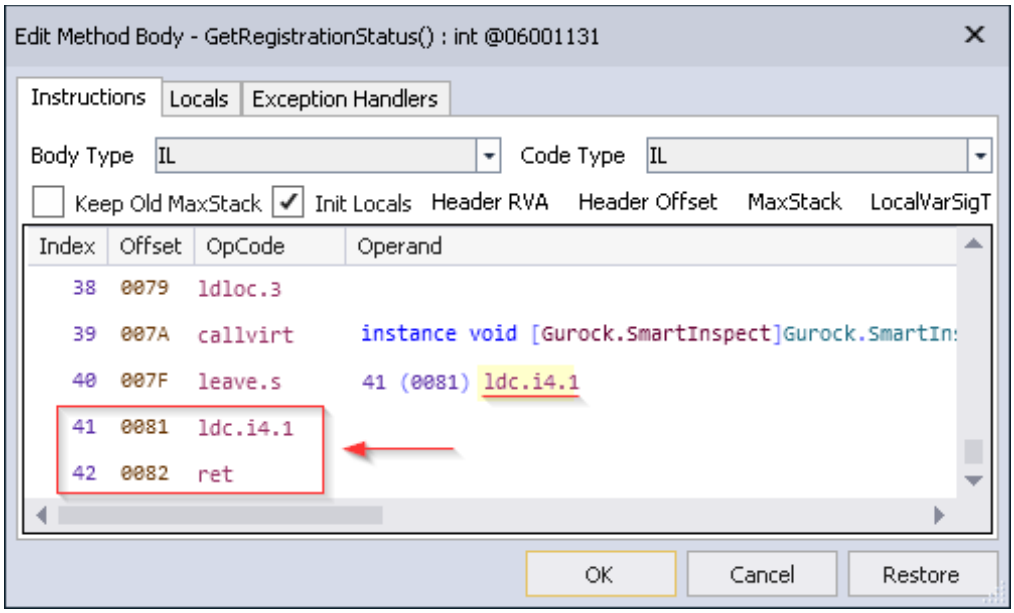


Veamos cómo quedó desde la ventana Locals:

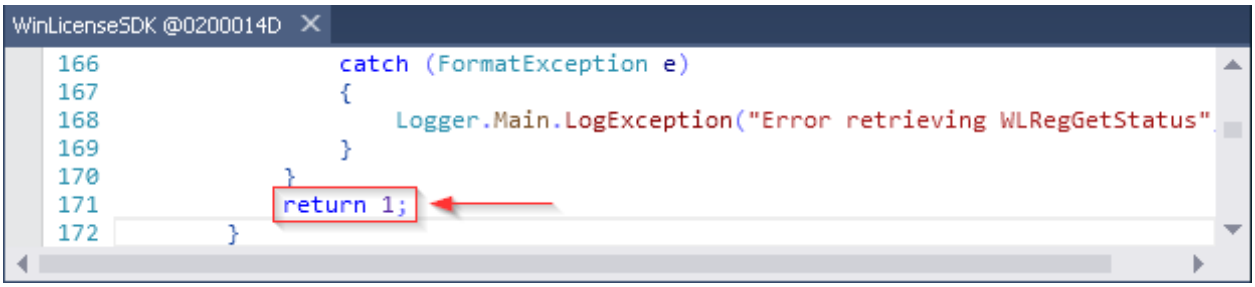
Perfecto, si seguimos con <F10> vemos que efectivamente el método retorna 0x01h. Es otra forma más de hacerlo, pero puestos a practicar esta vez vamos a modificar el ejecutable. Para esto necesitamos acceder al código IL que vimos antes, veamos el código del método GetRegistrationStatus():



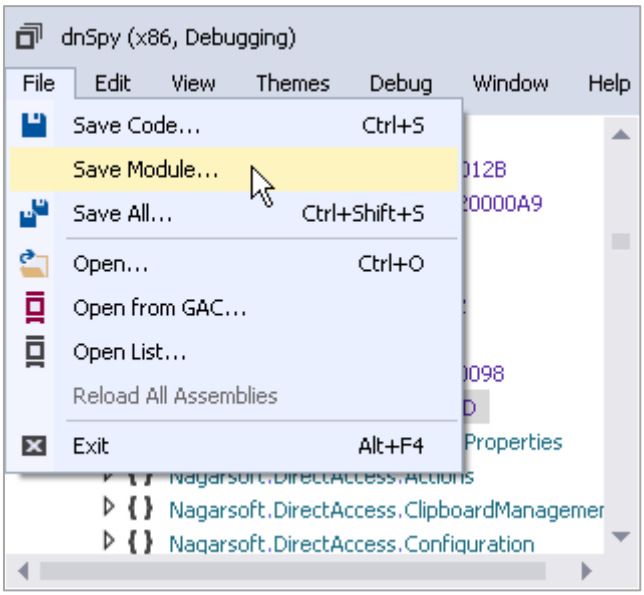
Ese es el retorno, hace `ldloc.2`, lo que significa que está subiendo la variable local con índice 2 en el array de Locals al stack, si en lugar de esto subiera un 0x01h retornaría 1, hagamos la prueba:



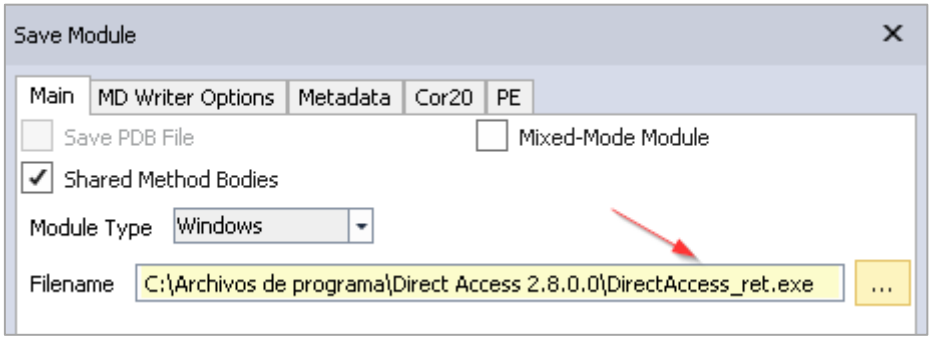
Pulsamos <Ok> y miramos cómo quedó el método:



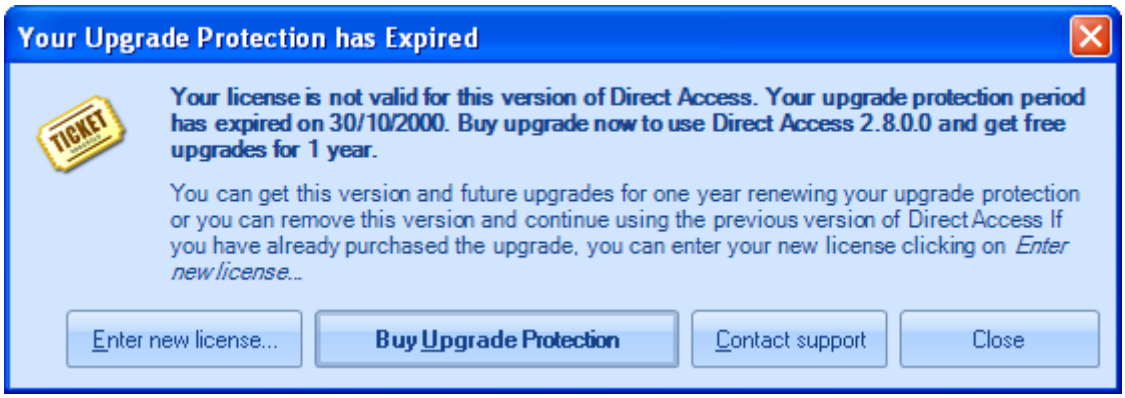
Fenomenal, salvamos los cambios al disco



Y le damos otro nombre, por ejemplo DirectAccess_ret.exe para saber que tiene modificado el valor de retorno de GetRegistrationStatus().



Si miramos de ejecutarlo veremos que funciona bien. Pero si atrasamos el reloj hasta 2000 por ejemplo sigue apareciendo la nag de EXPIRED:



Esto pasa porque hemos modificado únicamente un método, el resto de ellos continúan haciendo sus comprobaciones y demás, por lo que si algo no le cuadra te cierra el programa, como es el caso.

Lo que haremos será modificar también el punto donde se comprueba el reloj.

Vamos a la zona que tenemos identificada, es esta comparación:

```
Start @0200012A
556                 return false;
557             }
558         }
559         if (Start.buildDate > dateTime)
560         {
561             Logger.Main.LogMessage("License invalid for this version of the so
562             Start.RegistrationName = "<strong>" + Resources.UNREGISTEREDTRIAL
```

Una manera sencilla de pasar la comprobación es comparando la variable con ella misma, puesto que revisa si es MAYOR la comprobación NUNCA se cumplirá y se ejecutará SIEMPRE.

Nos situamos sobre la comparación y pulsamos <Ctrl + E> para editar el código IL:

```
Edit Method Body - VerifyLicense(out int) : bool @06001072
Instructions | Locals | Exception Handlers
Body Type: IL   Code Type: IL
[ ] Keep Old MaxStack  [x] Init Locals  Header RVA: 0x60E6  Header Offset: 0x5F06  MaxStack: 8  LocalVarSigTok: 0x110C

Index  Offset  OpCode     Operand
131    01B2    ret
132    01B3    ldsfld     valuetype [mscorlib]System.DateTime Nagarsoft.DirectAccess.Start::buildDate
133    01B8    ldloc.s    V_6 (6)
134    01BA    call       bool [mscorlib]System.DateTime::op_GreaterThan(valuetype [mscorlib]System.DateTime,
135    01BF    brfalse.s  166 (0235) ldloc.s V_7 (7)
136    01C1    call       class [Gurock.SmartInspect]Gurock.SmartInspect.Session Nagarsoft.DirectAccess.Logge

OK   Cancel   Restore
```

Ahí están identificadas las líneas que conforman la instrucción, veamos qué hacen:

S	Ldsfld	Pushes the value of a static field onto the evaluation stack.

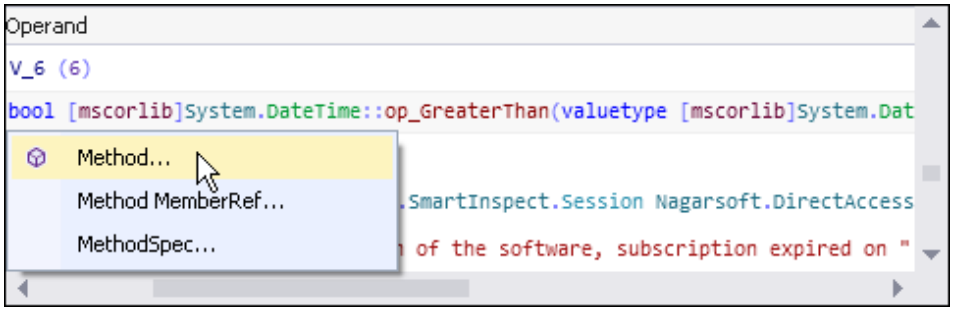
La primera sub-instrucción carga el valor de un campo *static* en el stack, si miramos la imagen anterior a la última (la penúltima) vemos que hace alusión al campo:

- ✓ Nagarsoft.DirectAccess.Start::buildDate.

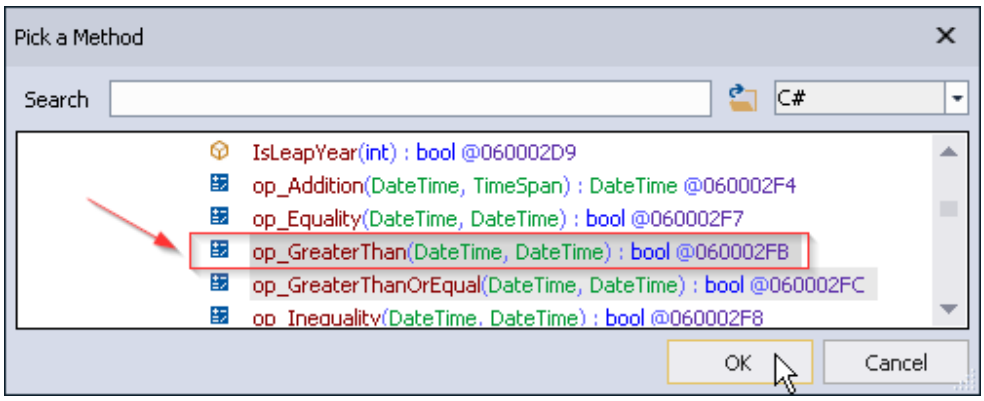
El segunda sub-instrucción carga el valor de la variable local con índice 6 en el array de Locals que corresponde con la variable dateTime, éstas son las variables que compara a través de la llamada que realiza en la tercera sub-instrucción:

- ✓ [mscorlib]System.DateTime::op_GreaterThan(valuetype [mscorlib]System.DateTime, …)

Y que podemos fácilmente modificar yendo a *Method…*:

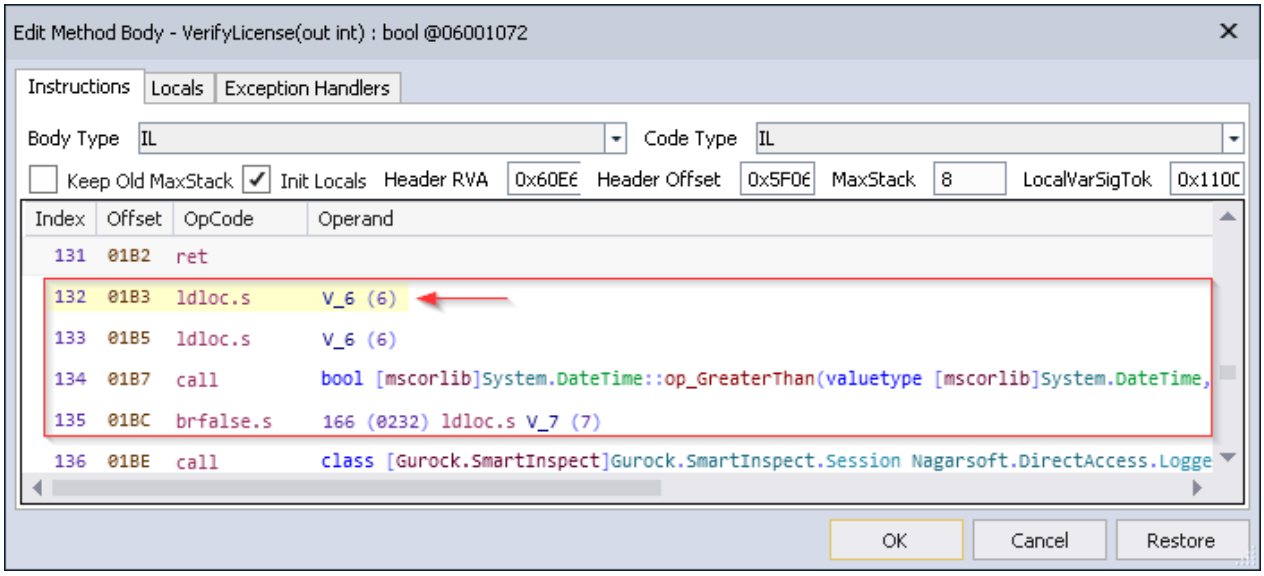


Y en el cuadro que se despliega seleccionando cualquier otro distinto del actual:



Seleccionando por ejemplo op_Equality la instrucción cambia:

Pero como estamos practicando cambiaremos directamente el código IL, lo que haremos será subir al stack el valor de la variable dateTime en lugar del campo static Start.buildDate:



Pulsamos <OK> y el cambio se aplica:

```
Start @0200012A
556                     return false;
557                 }
558             }
559             if (dateTime > dateTime)
560             {
561                 Logger.Main.LogMessage("License invalid for this version o
562                 Start.RegistrationName = "<strong>" + Resources.UNREGISTER
```

Como se puede ver, hay muchísimas formas distintas de modificar el código. Salvamos el cambio al disco y le damos otro nombre, por ejemplo DirectAccess_noExpira.exe, lo ejecutamos y funciona, atrasamos el reloj hasta el año 2000 y funciona, lo adelantamos a 2025 y también funciona independientemente del valor del reloj.

Aún así vamos a modificarlo de otra manera más elegante. Vayamos a la siguiente sección..

6. Todos los caminos llevan a Roma

Pues sí, así es, o al menos así era en tiempos Romanos donde la Vía Apia y la Vía Augusta atravesaban todo el continente, no obstante estar en 2015 y en otros tiempos hay cosas que no cambian.

Tenemos un ejecutable que ya funciona bien con independencia de las variables de entorno y del reloj, esto es muy bueno, pero metidos en materia nos sabe a poco. Si vamos al About vemos esto:

Esto es muy feo, además en la barra de estado dice Licensed To y al lado NO hay ningún nombre.

Pongámosle solución, vamos a trabajar de nuevo con DirectAccess_ret.exe que dejamos en el primer punto de la sección anterior.

Cargamos el Assembly DirectAccess_ret.exe en dnSpy (dotNetSpy ??), lo configuramos para que NO pare en el EntryPoint ni en el Constructor .cctor().

Esta vez nos centramos en el método GetRegistrationInfo().:

```
Start @0200012A
526        string text4;
527        DateTime dateTime;
528        bool flag2;
529        WinLicenseSDK.GetRegistrationInfo(registrationStatus, out arg, out
530        Start._serialNumber = text2.Trim();
531        Start.CustomerEmail = text3;
532        Logger.Main.LogString("SerialNumber", text2);
```

Este será motivo de análisis, entramos con <F11>:

```
WinLicenseSDK @0200014D
84         // Token: 0x06001137 RID: 4407 RVA: 0x00061F2C File Offset: 0x000
85         public static void GetRegistrationInfo(int status, out string nam
86         {
87             name = string.Empty;
88             company = string.Empty;
89             email = string.Empty;
90             numlic = string.Empty;
91             serialnumber = string.Empty;
92             subscribeExpireDate = DateTime.Today;
93             powerLicensePack = false;
```

Allí se ven las variables que utiliza este método, en realidad está asignando valor a los parámetros que recibe el método. sigamos traceando con <F10> hasta ver algo interesante como esto:

```
WinLicenseSDK @0200014D
92          subscribeExpireDate = DateTime.Today;
93          powerLicensePack = false;
94          string a = string.Empty;
95          string text = string.Empty;
96          if (status == 1 || status == 12)
97          {
98              StringBuilder stringBuilder = new StringBuilder(500);
99              WinLicenseSDK.GetEnvironmentVariable("WLRegGetLicenseInfo", stringBuilder, 500);
100             Logger.Main.LogString("WLRegGetLicenseInfo", stringBuilder.ToString());
101             string[] array = stringBuilder.ToString().Split(new char[]
102             {
103                 ','
104             });
```

Está buscando la segunda variable de entorno WLRegGetLicenseInfo y que no va a encontrar puesto que NO existe, pasamos la llamada con <F10> y miramos la ventana Locals:

Locals	
Name	Value
text	""
stringBuilder	{}
array	null
s	null

Efectivamente, no la encontró. No pasa nada, vamos a modificar el valor de la variable stringBuilder, hay que tener en cuenta que es un Objeto, expandimos la llave y cambiamos el valor m_StringValue:

Locals	
Name	Value
stringBuilder	{snat,CLS,123456-000;snat@gmail.com;2500;30001110;PowerLicense}
Capacity	0x0000003D
CapacityField	"Capacity"
DefaultCapacity	0x00000010
Length	0x0000003D
MaxCapacity	0x7FFFFFFF
MaxCapacityField	"m_MaxCapacity"
StringValueField	"m_StringValue"
ThreadIDField	"m_currentThread"
m_MaxCapacity	0x7FFFFFFF
m_StringValue	"snat,CLS,123456-000;snat@gmail.com;2500;30001110;PowerLicense"
m_currentThread	0x00198998
array	null

Al hacer esto el valor de la variable automáticamente se actualiza y muestra el contenido modificado, si dejamos correr el programa arranca algo mejor:

0 Commands | Licensed to snat (CLS) | Detect Abbreviations

Si vamos al About dnSpy para en el BP que tenemos puesto, repetimos la operación y vemos esto:

Se puede personalizar, estamos practicando así que TODO vale.

Nos queda hacer el cambio EFECTIVO en el EXE, puesto que lo hemos hecho al vuelo, trataremos de no repetir instrucción y utilizar otra que aún no hayamos utilizado, veamos esta otra:

S	Ldstr	Pushes a new object reference to a string literal stored in the metadata.

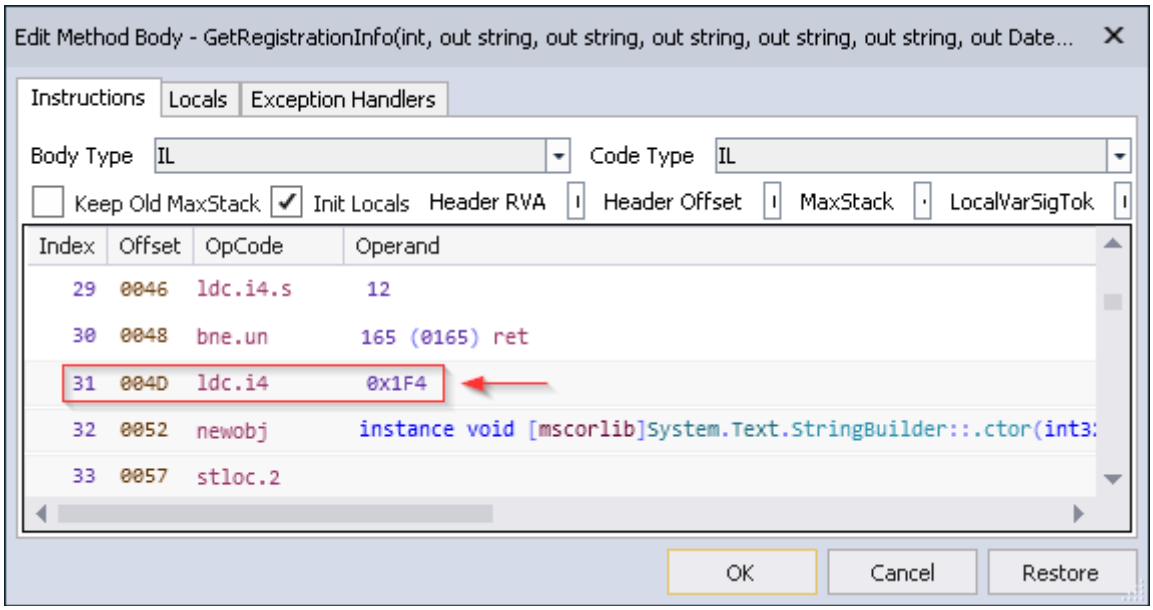
Se utiliza para cargar strings, pero dónde? La respuesta está en la creación del Objeto StringBuilder:

- ✓ StringBuilder stringBuilder = new StringBuilder(500);

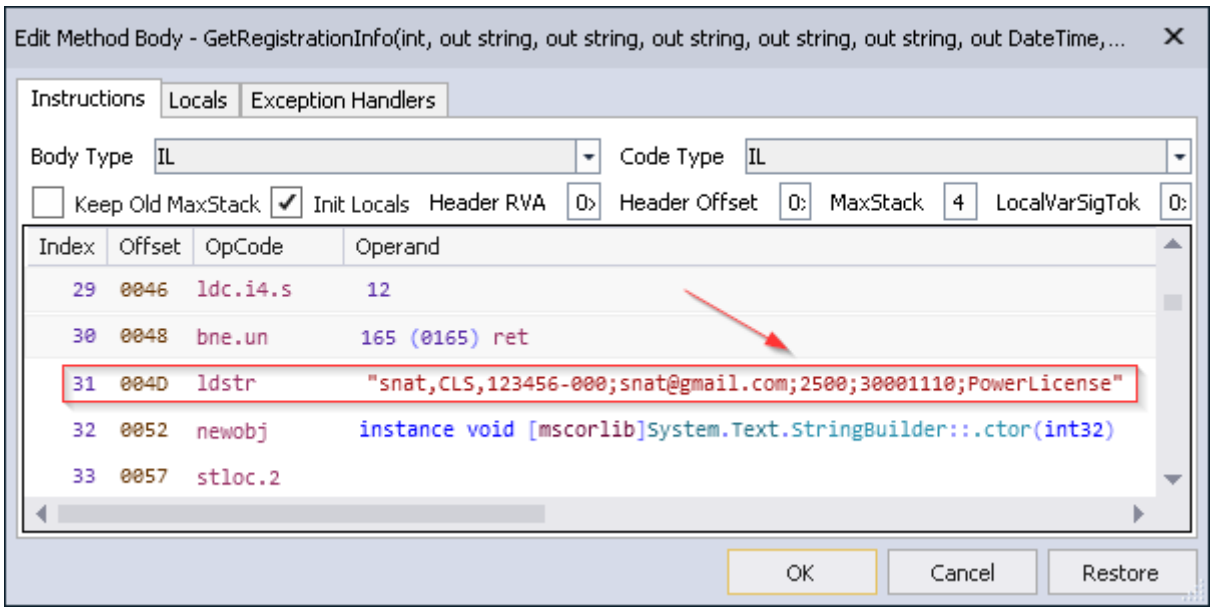
En su lugar haremos esto:

- ✓ StringBuilder stringBuilder = new StringBuilder("NUESTRA CADENA");

Sustituyendo "NUESTRA CADENA" por el que queramos, editamos el código IL <Ctrl + E> y cambiamos:



Que es la reserva de 0x1F4h (500d) bytes, por esto:



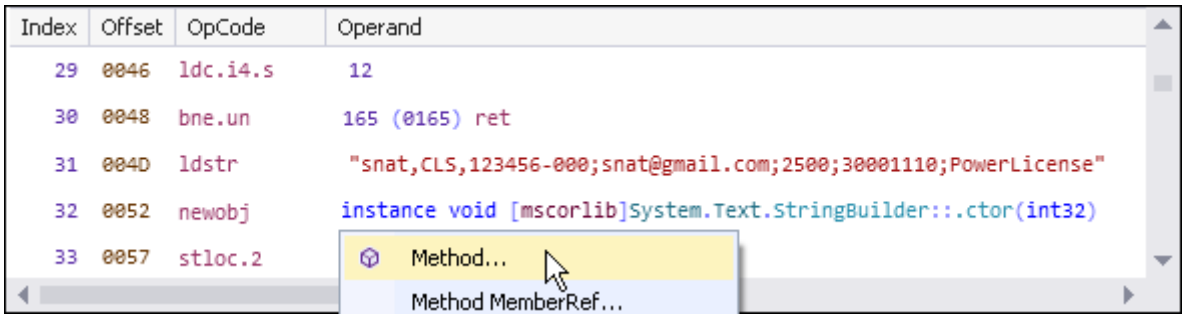
Si salvamos los cambios a disco y lo probamos NO funciona!, el problema está en el constructor de Clase:

- System.Text.StringBuilder::.ctor(int32)

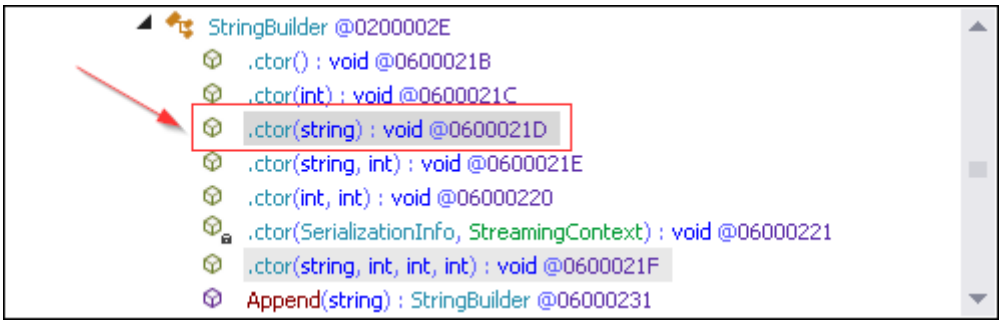
Que hay que modificar a:

- System.Text.StringBuilder::.ctor(string)

Lo modificamos seleccionando la opción *Method*:



Y seleccionamos el nuevo Constructor con parámetro string:

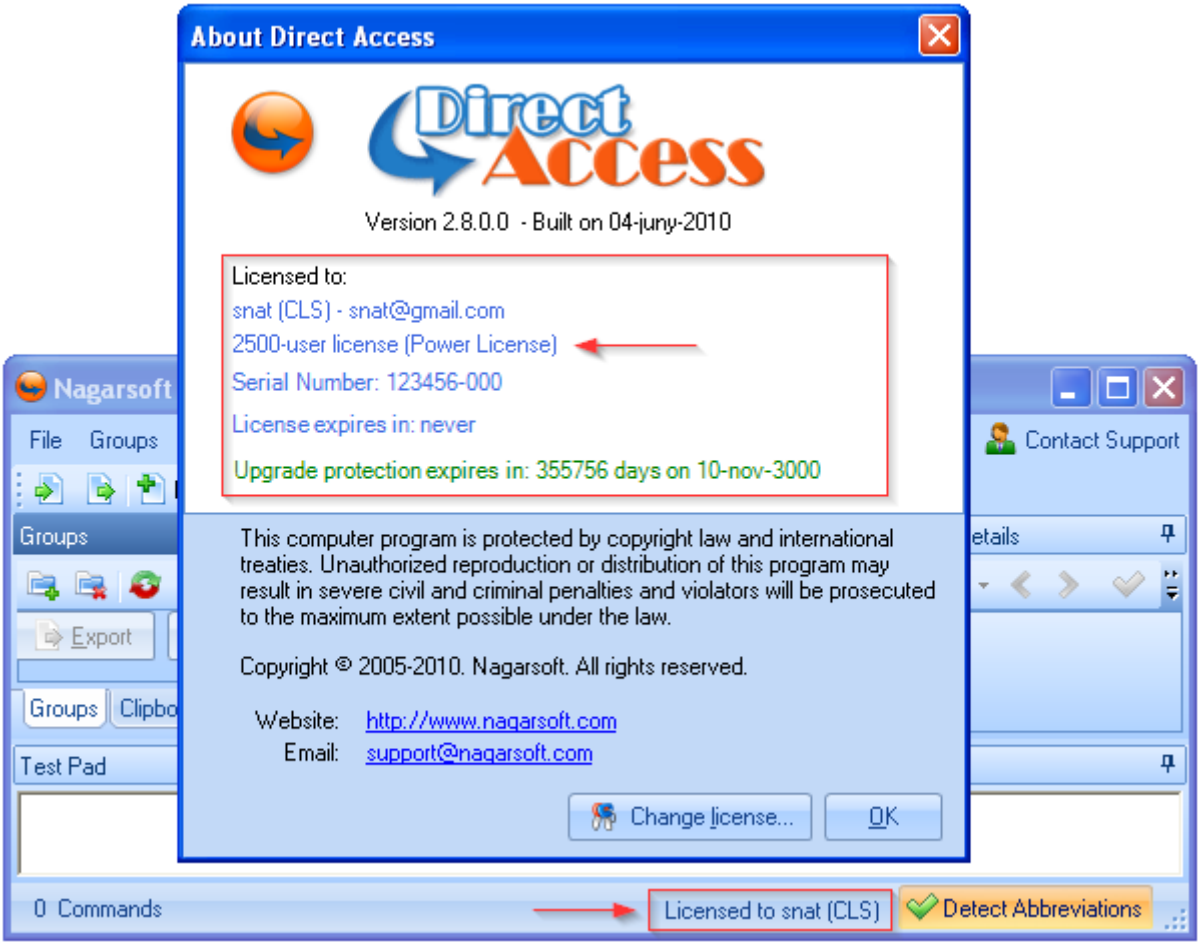


Y aceptamos <OK> el cambio deberá aparecer reflejado.

Allí está:

Index	Offset	OpCode	Operand
29	0046	ldc.i4.s	12
30	0048	bne.un	165 (0165) ret
31	004D	ldstr	"snat,CLS,123456-000;snat@gmail.com;2500;30001110;PowerLicense"
32	0052	newobj	instance void [mscorlib]System.Text.StringBuilder::.ctor(string)
33	0057	stloc.2	

Salvamos el cambio al EXE DirectAccess_final.exe y lo probamos:



Ahora SI =)

Podemos adelantar el reloj tanto cuanto queramos y atrasarlo que sigue funcionando por lo que nuestro trabajo con el Assembly ha finalizado.

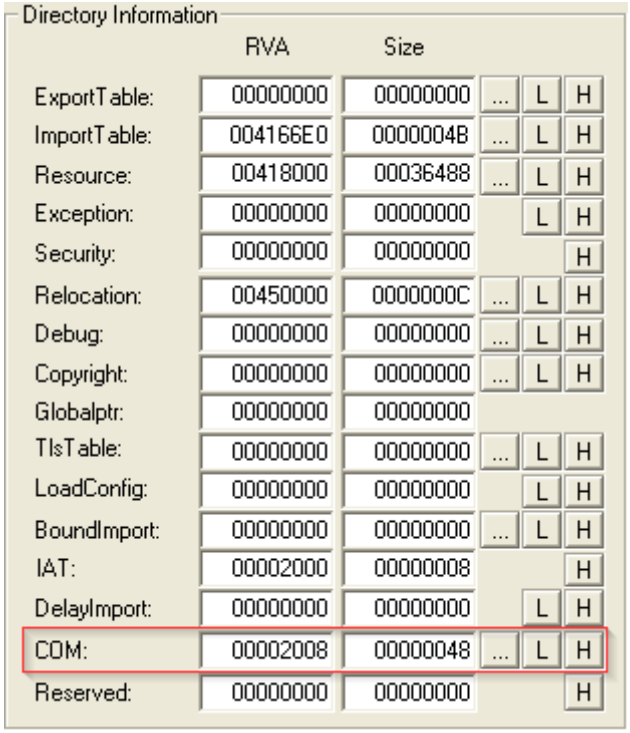
7. En busca de los Metadatos perdidos

Pues sí, eso es y no nos vamos a despedir sin antes hacer alguna práctica extra. En el texto de Themida en .NET me quedé con la intriga de saber donde guardaban los Assembly's su tan preciado tesoro: Los METADATOS. Sabemos por la teoría que en ellos se describe información tal como:

- Identidad: nombre, versión, cultura, llave pública.
- Los tipos exportados.
- Otras dependencias entre assembly's.
- Permisos de seguridad necesarios para la ejecución.

En definitiva, contiene TODA la información necesaria para que el Assembly se pueda ejecutar como tal y para que pueda interactuar con su entorno.

Estos metadatos son creados por el compilador, a la par que genera el código IL (también conocido como MSIL) y son insertados en el propio Assembly. Si abrimos LordPE que lo teníamos reservado precisamente para esto podemos encontrar estos metadatos en el 15º directorio dentro del DATA_DIRECTORY bajo el nombre COM (Common Object Model):



Directory Information

	RVA	Size			
ExportTable:	00000000	00000000	...	L	H
ImportTable:	004166E0	0000004B	...	L	H
Resource:	00418000	00036488	...	L	H
Exception:	00000000	00000000		L	H
Security:	00000000	00000000			H
Relocation:	00450000	0000000C	...	L	H
Debug:	00000000	00000000	...	L	H
Copyright:	00000000	00000000	...	L	H
Globalptr:	00000000	00000000			
TlsTable:	00000000	00000000	...	L	H
LoadConfig:	00000000	00000000		L	H
BoundImport:	00000000	00000000	...	L	H
IAT:	00002000	00000008			H
DelayImport:	00000000	00000000		L	H
COM:	00002008	00000048	...	L	H
Reserved:	00000000	00000000			H

¡Ya podía yo buscar! mira que busqué.

Estamos en el Descriptor COM. Desde aquí podemos encontrar TODA la información que necesitemos del Assembly: basta con desplazarse por el mismo.

Hagamos una prueba, abramos DirectAccess_final.exe en OllyDbg, y antes de hacer nada en LordPE vamos al botón de la L del Directorio COM:

```
[ Structure Lister ]
->COM Descriptor
   cb:                      0x00000048
   MajorRuntimeVersion:     0x0002
   MinorRuntimeVersion:     0x0005   -> 2.05
   Flags:                   0x00000003   (ILONLY, ILONLY)

   DataDirectory                RVA         Size
   -------------                ----------  ----------
   MetaData                     0x00395EC0  0x00080820
   Resources                    0x000621C8  0x00333CF8
   StrongNameSignature          0x00000000  0x00000000
   CodeManagerTable             0x00000000  0x00000000
   VTableFixups                 0x00000000  0x00000000
   ExportAddressTableJumps      0x00000000  0x00000000
   ManagedNativeHeader          0x00000000  0x00000000
                                                        OK
```

El campo Metadata es el que nos interesa por ahora. Partimos del offset 0x00002008h, tenemos un ImageBase de 0x11000000h, calculemos:

VA = 0x11000000h + 0x00002008h == 0x11002008h

Vayamos a esta dirección en Olly:

```
Address   Hex dump                                          ASCII
11002008  48 00 00 00 02 00 05 00 C0 5E 39 00 20 08 08 00  H........^9. ...
11002018  03 00 00 00 61 10 00 06 C8 2[illegible] 06 00 F8 3C 33 00  ....a....↑...<3.
11002028  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
11002038  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
11002048  00 00 00 00 00 00 00 00 0A 00 0D 00 20 00 00 00  ............ ...
11002058  00 01 01 81 36 02 28 3C 00 00 0A 02 28 0B 00 00  ....6.(<....(...
11002068  06 2A 7A 03 2C 13 02 7B 02 00 00 04 2C 0B 02 7B  .*z.,..{....,..{
11002078  02 00 00 04 6F 1A 00 00 0A 02 03 28 3D 00 00 0A  ....o......(=...
```

Ahí está el offset del MetaDirectory, calculemos:

VA = 0x11000000h + 0x00395EC0h == 0x11395EC0h

Vayamos hasta allí:

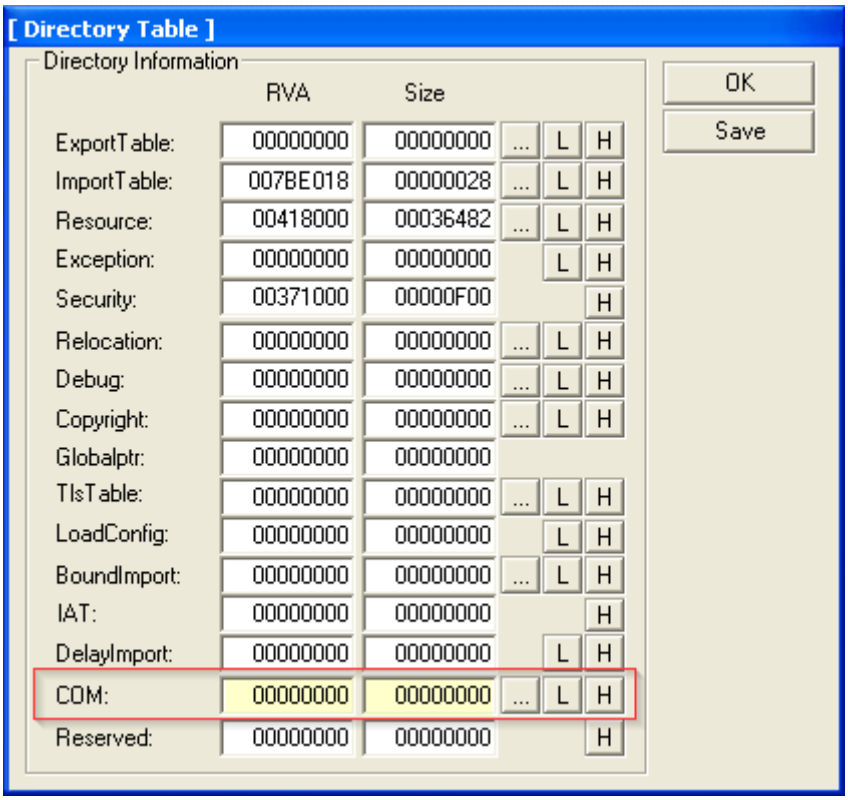
```
Address   Hex dump                                          ASCII
11395EC0  42 53 4A 42 01 00 01 00 00 00 00 00 0C 00 00 00  BSJB............
11395ED0  76 32 2E 30 2E 35 30 37 32 37 00 00 00 00 05 00  v2.0.50727......
11395EE0  6C 00 00 00 90 14 03 00 23 7E 00 00 FC 14 03 00  l.......#~......
11395EF0  0C 88 02 00 23 53 74 72 69 6E 67 73 00 00 00 00  ....#Strings....
11395F00  08 9D 05 00 0C E1 01 00 23 55 53 00 14 7E 07 00  ........#US..~..
11395F10  10 00 00 00 23 47 55 49 44 00 00 00 24 7E 07 00  ....#GUID...$~..
11395F20  FC 89 00 00 23 42 6C 6F 62 00 00 00 00 00 00 00  ....#Blob.......
11395F30  02 00 01 01 57 BF B7 3F 09 0B 00 00 00 FA 25 33  ....W..?......%3
```

Allí está la versión v2.0.50727 que buscaba en la llamada a GetFileVersion:

```
Address   Hex dump    Disassembly                          Registers (FPU)
603BB255    E8 09380000  call    603BEA63                  EAX 0017B690 UNICODE "v2.0.50727"
603BB25A    8985 8CFBFFFF  mov     dword ptr [ebp-0x474], eax   ECX 603BEB01
603BB260    89B5 90FBFFFF  mov     dword ptr [ebp-0x470], esi   EDX 00180002
603BB266    3BC6         cmp     eax, esi                  EBX 0012FCC4
603BB268  v 74 0A        je      short 603BB274            ESP 0012F768
603BB26A    C785 90FBFFFF  mov     dword ptr [ebp-0x470], 0x1   EBP 0012FC54
603BB274    C745 FC 410000 mov     dword ptr [ebp-0x4], 0x41    ESI 00000000
603BB27B    8B85 8CFBFFFF  mov     eax, dword ptr [ebp-0x474]   EDI 6040C090
603BB281    3BC6         cmp     eax, esi                  EIP 603BB25A
eax=0017B690, (UNICODE "v2.0.50727")
```

Recordemos además que en el primer dumpeado no encontraba este valor y retornaba EAX==0x00h.

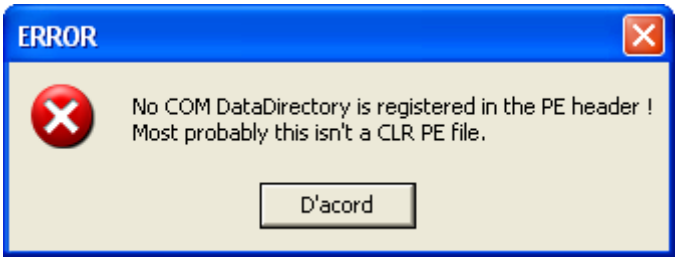
Veamos por qué. Abrámoslo con LordPE y vayamos al 15º directorio del OPTIONAL_HEADER:



Está a 0x00h, vayamos al botoncito de la L:

```
->COM Descriptor
   Not present !
```

Si miramos con el botón de los puntos suspensivos:



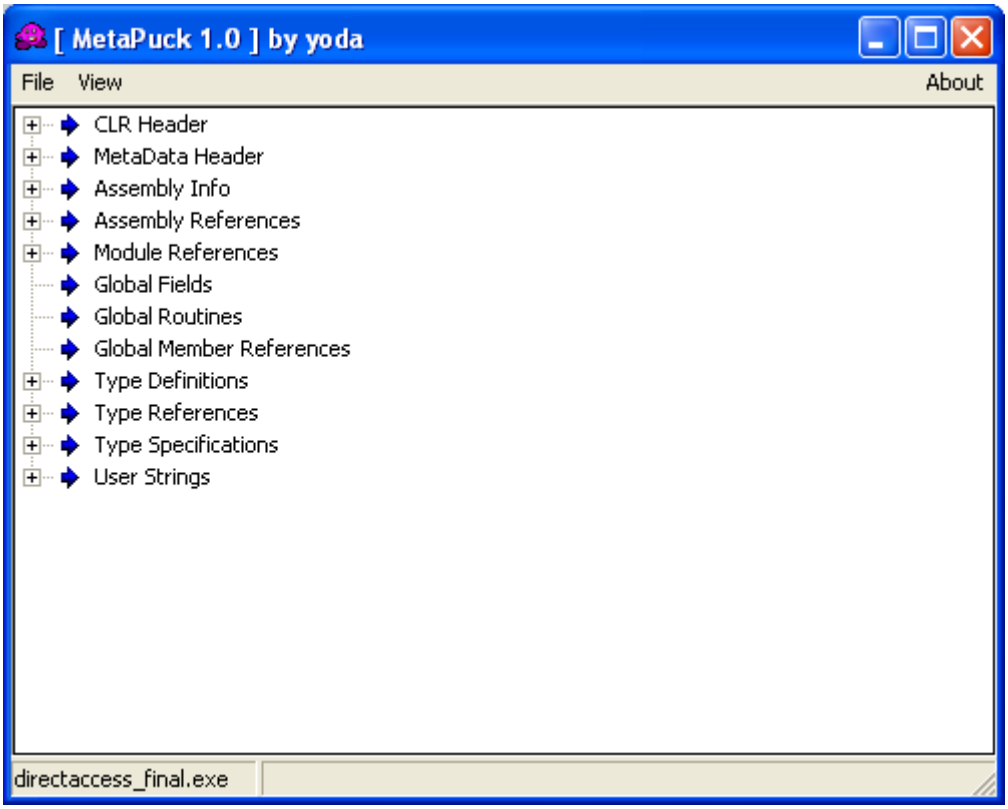
Si lo intentamos abrir con dnSpy nos dice esto:

// This file does not contain a managed assembly.

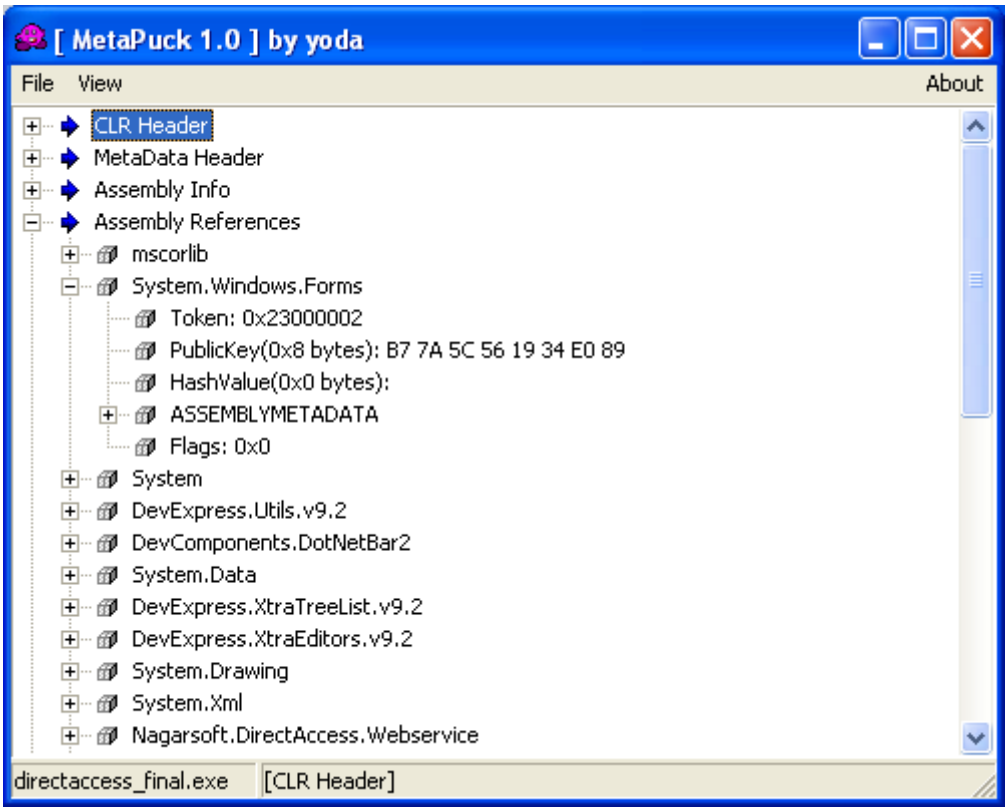
Por tanto, era IMPOSIBLE que el dumpeado se pudiera ejecutar, LordPE no fue capaz, esta vez, de darnos un dumpeado válido, no se lo tendremos en cuenta por TODO lo que ha hecho por nosotros. Pero SI que es cierto que NUNCA hubiéramos podido ejecutar el Assembly sin sus METADATOS.

De errores se aprende :(

En cambio, si vamos a los puntos suspensivos para DirectAccess_final.exe cuyo dumpeado, recordemos, nos dio MegaDumper v1.0 de SnD vemos que allí SI están los valores correctos:

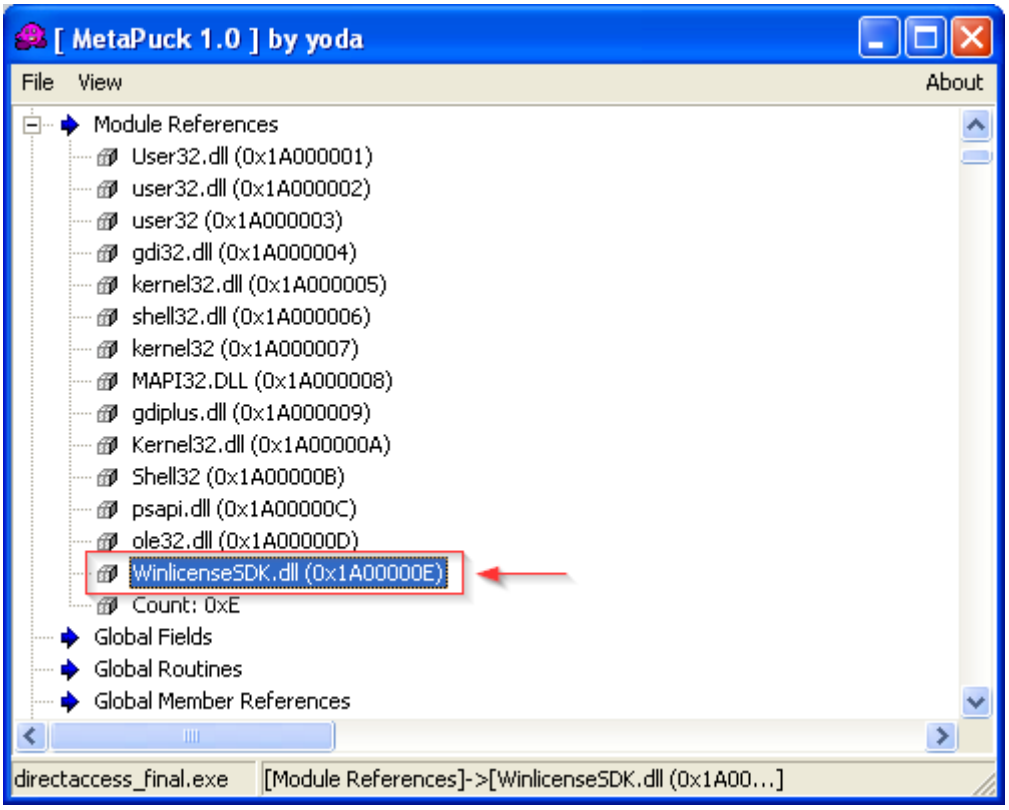


Si desplegamos una cualquiera vemos su información:



Es de VITAL importancia preservarla para el correcto funcionamiento del Assembly, no cabe duda.

Todavía hay muchísima más:



Sin estos datos poco podíamos hacer.

Y Ahora que hemos visto donde encontrar los METADATOS podemos hacer un punto y aparte en la investigación de los .NET.

8. Notas Finales

Un tema apasionante y didáctico. Hay muchos otros temas por cubrir en lo relacionado con el lenguaje .NET que podrían ser cubiertos perfectamente en un texto aparte dedicado a esta plataforma, que en mi opinión, todavía tiene cuerda para darnos.

El texto se ha intentado centrar más bien en aspectos relativos a dnSpy, una herramienta que podemos considerar como MUY BUENA, en un ranking de 1 a 5 le podríamos dar 4,5 tranquilamente. Por poner un pero he de decir que he echado en falta al menos estas dos características:

- Accesibilidad a variables de Clase (static)
- Colocación de BreakPoints on Memory

Quizá estas opciones estén ahí y no las he sabido encontrar ¿???

De todos modos la valoración sigue siendo MUY ALTA y es una herramienta que nos va a permitir llegar al corazón de los .NET hasta ahora reservado para gente con bastante conocimiento de éstos. Por lo que no quiero desaprovechar la oportunidad de agradecer a mi amigo sequeyo su GRAN aportación.

Respecto al texto, he intentado mostrar lo que he ido viendo de manera sencilla y clara, la idea es que éste se pueda seguir con facilidad.

Por todo lo demás sólo me queda AGRADECER a las personas que desarrollan herramientas como esta, así como a aquéllos que dedican parte de su tiempo en escribir para que los demás aprendamos y sigamos avanzando. Para mí ha sido un placer el poder haber escrito estas líneas y espero poder escribir otras tantas. Un gran saludo para TODOS los CrackSLatinos de los que cada día se aprende algo nuevo :DD

Hasta la próxima..

CrackSLatinoS
Snat.~